SeinARTS Framework — Design Reference

A living reference for the design of SeinARTS's core simulation primitives and their interactions. Not a roadmap — a "how does this system work and why" document that stays accurate as implementation evolves.

Audience: the designer/author (RJ), future contributors, future Claude sessions. Paired with CLAUDE.md (orientation-for-assistants) and the code itself (source of truth).

Scope: the sim-layer primitives and the systems that span them. Render/editor/UI concerns appear only where they cross the sim boundary.

Design target: the RTS genre across its typical spectrum — from small-squad tactical games to massive-unit economy-flow games, including resource-gathering, tech-progression, cover-based combat, formation tactics, and hybrid subgenres. Most shared DNA, variance at the edges.

Framework generality vs. specific-subgenre bias. The framework is designed to be genre-neutral within the RTS space — anyone building a multiplayer RTS in Unreal should be able to ship with it. Starter content matching specific RTS subgenres (squad-tactical with directional cover, hero-joins-regiment formations, economy-flow upkeep models, etc.) ships as optional assets under clearly-labeled namespaces like `SeinARTS.Starter.SquadTactics.*`. Framework core never depends on them; designers use, extend, replace, or ignore them as the game demands. Same pattern as `FSeinWeaponProfile` / `FSeinCapturePointData` starters — aid, not dependency.

Status legend

Each section carries a status tag so we know where in the design pass it is:

- `[open]` — no decisions yet, placeholder
- `[drafting]` — Q&A in progress, decisions accumulating
- `[locked]` — design decisions settled; implementation may or may not be done
- `[re-opened]` — decisions being revisited; previous contents archived inline

---

Cross-cutting invariants `[locked]`

Every primitive and system in this document must satisfy these. A design that fights them is rejected.

1. **Sim/render separation is absolute.** The simulation runs on fixed-point math (`FFixedPoint`, `FFixedVector`, etc.) and deterministic primitives. It never reads `AActor*`, `FVector`, `FMath::`, `float` field values, wall-clock time, or non-deterministic RNG. The render/editor/UI layer reads from the sim and reacts to visual events. Data flows one way: sim → render.
2. **Lockstep-compatible.** Only `FSeinCommand` entries cross the network. All sim state is derived from the sequence of commands applied to a known initial state. Clients stay bit-identical tick-by-tick. Observer-only commands (camera moves, selection changes) may enter the command log for replay reconstruction but produce no sim-state mutation.
3. **Replay-serializable.** A match replay is a command stream plus an initial RNG seed. Replaying feeds commands at their recorded ticks and re-runs the sim. State-hash checks at each tick validate determinism. This implies: no primitive stores transient non-deterministic data; every piece of sim state is either re-derivable from commands or serializable into a frame snapshot.
4. **Tick phase must be declared.** Every system that mutates sim state runs at a specific phase (PreTick / CommandProcessing / AbilityExecution / PostTick) with a known priority. No ad-hoc mutation from outside

the tick pipeline. Exception: `ProcessCommands` runs inside CommandProcessing and dispatches ability activations synchronously.

5. **Tag integration.** When a system exposes state that other systems might want to query positionally or conditionally, it should expose that state as tags on the entity (via `FSeinTagData`). Other systems read tags, not direct component fields. Keeps coupling loose and designer-editable queries possible.
6. **No premature replication abstractions.** We are building a deterministic lockstep framework, not a traditional client-server replication framework. Replication lives in Unreal's built-in networking stack for lobby/chat/handshake only. Sim state is never replicated via `UPROPERTY(Replicated)` because it's deterministically recomputed.
7. **Designer-editable where possible.** If a field can reasonably be authored in Blueprint defaults rather than hardcoded in C++, it should be. The framework provides primitives; the game's content authors compose them.
8. **Scale-aware grid-backed data.** Grid-backed systems (nav, vision, terrain, spatial queries) must support maps up to ~5000^2 cells. Per-cell structures stay compact (4-8 bytes per cell, not 30+). Tag data uses per-map palettes with u8 indices + sparse overflow for rare cases. Tile-based spatial partitioning (16×16 or 32×32 cells per tile, configurable) is standard infrastructure for range queries and sparsity skipping. Cell sizes are designer-configurable — small-scale squad-tactical games use 1 m cells (~1000^2-2000^2 grids); massive-unit games use 8-16 m cells (~1000^2-5000^2 grids). Framework practical ceiling is ~5000^2 cells regardless of physical map size; designers choose cell size to match unit scale, not map scale.
9. **Batch-tick heavy systems.** Systems doing expensive work (vision stamping, capture polling, large-radius spatial scans, flow-field recomputation) declare a `TickInterval` on `ISeinSystem` and run every N sim ticks. Default 4-tick batching for heavy work (7.5 Hz at 30 Hz sim — below human latency threshold for non-critical feedback). Only latency-sensitive systems (command processing, ability ticks, damage resolution) tick every frame. All clients batch on identical ticks; determinism preserved.
10. **Entity lookup is framework-DNA-level.** `USeinWorldSubsystem` maintains two global indices updated automatically by the tag-refcount system (§2) and explicit registration: a `TMap<FGameplayTag, TArray<FSeinEntityHandle>>` auto-indexed by entity tags (every grant/ungrant updates it), and a `TMap<FName, FSeinEntityHandle>` for singleton named lookups. Tag-based queries ("all Riflemen", "all stealthed enemies in radius") are O(1) TMap lookups, not O(entities) iterations. Named lookups ("the scenario", "HeroBob") are O(1) direct fetches. Every grid-backed or entity-iterating system should prefer these lookups over manual filter-sweeps.

---

1. Entities `[locked]`

The atomic unit of sim existence. Anything with sim-side state.

Decisions

- **Definition.** Entity = anything with sim state. Position is optional — `FFixedTransform` is a default field on every entity slot, but abstract entities (squads, scenario-owners, future purely-stateful entities) simply ignore it. No second primitive for "abstract entity"; the one-pool model wins on simplicity.
- **Required component set.** Every entity implicitly carries `FSeinTagData` in component storage. Tag queries are cross-cutting infrastructure (ability arbitration, attribute resolution, terrain checks) — forgetting to add tag storage is always a bug, never intent. No separate `USeinTagsComponent` authoring surface.
- **Initial tags live on the archetype def.** `USeinArchetypeDefinition` gains a `BaseTags` field. At spawn, the auto-injected `FSeinTagData` is initialized from it. Designers author initial unit tags in one place alongside identity, cost, tech metadata, and default commands — the archetype def is the complete per-unit-type authoring surface.
- **Ownership.** Every entity has a resolvable owner `FSeinPlayerID`. Entities that logically have "no player"

(neutral capture points, resource piles, environmental hazards) are owned by a dedicated **`Neutral`** player registered at match start alongside human/AI players. No sentinel `None` owner — one codepath beats marginal semantic precision.

- **Short-lived entities** (projectiles, damage zones, decoys, sim-side VFX anchors) share the main entity pool. An optional `FSeinLifespanData { int32 ExpiresAtTick; }` component auto-reaps them in `PostTick`. Tag classification (`Entity.Transient`) makes them queryable without a separate pool.
- **Handle stability.** `FSeinEntityHandle` generation counters persist across save/load. Save/load is not "snapshot state" — it's "replay the command stream from tick 0 to this tick." The command log is the save system. Designers can later serialize state via Unreal tools for crash-recovery or cross-version compatibility, but the canonical save is the command stream.

Non-goals

- Separate entity pool or separate primitive for projectiles. Pool pressure is not currently a bottleneck; revisit if profiling says so.
- Sentinel `None` owner. If a genuine use case surfaces that `Neutral` can't cover, add it as a follow-up.
- Snapshot-based save/load. Command-stream replay is authoritative.

Implementation deltas that fall out

- Delete `USeinTagsComponent` (the render-side AC wrapper). `FSeinTagData` stays as the sim-side payload.
- Add `FGameplayTagContainer BaseTags` field to `USeinArchetypeDefinition`.
- `SpawnEntity` always creates `FSeinTagData` for the entity, initialized with `BaseTags` from the archetype def + the `ArchetypeTag` applied + `CombinedTags` rebuilt.
- Add `FSeinLifespanData` struct + `FSeinLifespanSystem` (PostTick) that reaps entities whose `ExpiresAtTick <= CurrentTick`.
- Register a `Neutral` player automatically at sim start so every entity has a resolvable owner.

---

2. Tags `[locked]`

Gameplay tags on entities. Used for arbitration, queries, classification, and cross-system communication. Every entity carries `FSeinTagData` (per §1).

Decisions

- **`BaseTags` is runtime-mutable.** Convention: `BaseTags` carries archetype-class identity (e.g., `Unit.Infantry`, `Unit.Vehicle`); `GrantedTags` carries runtime coordination state (anim sync, transient flags). A unit that morphs class (infantry → mech suit changes `Unit.Infantry` → `Unit.Vehicle`) mutates `BaseTags`. `FSeinTagBPFL` gains Add/Remove/Replace ops for `BaseTags`.
- **Refcounted tag presence.** `FSeinTagData` carries a per-tag refcount map. A tag is "present" (i.e., appears in `CombinedTags`) if at least one source still references it. Multiple abilities/effects/components can grant the same tag safely; the tag only disappears when the last source releases it.
- **Global tag → entity index, auto-maintained.** `USeinWorldSubsystem` holds `TMap<FGameplayTag, TArray<FSeinEntityHandle>> EntityTagIndex`. Every grant (0→1 refcount edge) appends the entity's handle to the bucket; every ungrant (1→0) removes it. Entity destroy iterates the entity's tags and removes from each bucket. This gives O(1) tag-based entity queries across the whole match — designers writing "all entities with tag X" become trivial BPFL lookups instead of iterate-all-entities-and-filter. Both `BaseTags` and `GrantedTags` auto-index (any tag a refcount goes non-zero for enters the index). Memory trivial at any RTS scale (4 KB – 400 KB total for typical matches). Insertion-order iteration is deterministic. Promoted to a framework-DNA invariant (preamble #10) because so many patterns use it.

- **No source attribution beyond refcount.** Option (b) from Q7, not (c). Refcounts handle correctness; debugging overlapping sources can be done via logs if needed. Revisit if debugging pain grows.
- **Component-granted tags auto-strip on component removal.** Falls out of refcounting: a component removal decrements whatever refcounts it contributed to. Mid-life component removal is currently rare, but this is the correct protocol when we get there. No implementation work required today beyond routing component-removal through the refcount path.
- **`ArchetypeTag` is a soft identifier.** Most games will use strict per-archetype tags (e.g., `Unit.M8Greyhound`) because modifiers usually target a specific archetype. But the framework allows multiple Blueprints to share one `ArchetypeTag` (e.g., `Unit.Infantry.Sniper` shared by Allies_Sniper and Axis_Sniper) for cross-faction archetype modifiers. Both usages are valid.
- **Framework-shipped tags use the `SeinARTS.*` namespace.** These are the framework's owned vocabulary (`Ability.Move`, `State.Channeling`, `CommandContext.RightClickGround`, `Tech.*` if we ship baseline tech, etc.). Defined in `SeinARTSGameplayTags.h/cpp`. Designers use them as-is when wiring framework-shipped abilities/commands.
- **Game-specific tag conventions are recommended, not enforced.** The framework leads by example in its own namespace; game authors are free to follow the suggested hierarchy below or invent their own.

Suggested hierarchy (non-binding — designers may adopt, adapt, or ignore)

```
SeinARTS.Unit.<Class>[.<Subtype>]         // Unit.Infantry.Sniper, Unit.Vehicle.Tank
SeinARTS.Ability.<Name>                   // Ability.Move, Ability.Sprint
SeinARTS.State.<Name>                     // State.Channeling, State.Stealthed
SeinARTS.CommandContext.<Context>         // CommandContext.RightClickGround
SeinARTS.Terrain.<Category>.<Subcategory> // Terrain.Cover.Heavy
SeinARTS.Resource.<Name>                  // Resource.Munitions
SeinARTS.Tech.<Name>                      // Tech.EfficientProduction
SeinARTS.Entity.<Modifier>                // Entity.Transient, Entity.Garrisoned
```

Non-goals

- Per-source attribution of tag grants beyond refcount.
- Edit-time or spawn-time validation of tag paths against a framework schema.
- Enforced tag hierarchy on game-specific content.

Implementation deltas that fall out

- Add `TMap<FGameplayTag, int32> TagRefCounts` to `FSeinTagData`.
- `GrantTag(tag)` increments refcount, adds to `CombinedTags` on 0→1 transition, **appends handle to `USeinWorldSubsystem::EntityTagIndex[tag]`** on same 0→1 edge.
- `UngrantTag(tag)` decrements refcount, removes from `CombinedTags` on 1→0 transition, **removes handle from `EntityTagIndex[tag]`** on same 1→0 edge.
- On entity destroy: iterate entity's `TagRefCounts` keys, remove handle from each tag's index bucket.
- Add `USeinWorldSubsystem::EntityTagIndex: TMap<FGameplayTag, TArray<FSeinEntityHandle>>` + `NamedEntityRegistry: TMap<FName, FSeinEntityHandle>`.
- `USeinEntityLookupBPFL` (framework-level): `SeinRegisterNamedEntity` / `SeinLookupNamedEntity` / `SeinUnregisterNamedEntity` / `SeinLookupEntitiesByTag` / `SeinLookupFirstEntityByTag` / `SeinCountEntitiesByTag` / `SeinHasAnyEntityWithTag` / `SeinLookupEntitiesByTagInRadius` / `SeinLookupEntitiesByTagForPlayer` / `SeinLookupEntitiesMatchingTagQuery`.
- `FSeinTagBPFL` gains mutation ops for `BaseTags` (Add / Remove / Replace).
- Remove `AppliedOwnedTags` runtime field from `USeinAbility` — refcount model subsumes the diff-on-activate workaround. `ActivateAbility` does a plain `GrantTag` per entry in `OwnedTags`; `DeactivateAbility` does a plain `UngrantTag` per entry. Archetype-granted tags stay present because they hold an independent

refcount.
- Retire the diff-apply comment in `USeinAbility::ActivateAbility` (we just simplified it).

---

3. Commands `[locked]`

Player/AI intent crossing the wire. A command is an entry in the lockstep txn log; all clients deterministically re-derive sim state from the command stream.

See also: **§4 CommandBrokers** (the sim-side primitive that mediates commands to dynamic member sets).

Decisions

- **Txn log carries pre-resolution intent.** The wire-format for a player click is `"Right-click at X with target Y by player Z"` — not `"Activate ability Move on entity E"`. Every client's sim runs the same deterministic resolver on the same state and ends up at the same ActivateAbility call. Keeps the wire format minimal and resolution policy changes (e.g., adding a smart-command context) replay-compatible.
- **Local debug output is separate from the txn log.** The current in-editor command log overlay shows post-resolution `ActivateAbility [tag]` lines. That's debug visibility, not wire-format. Belt + suspenders at zero cost — the wire stays minimal, designers still see resolved actions.
- **CommandType uses gameplay tags, not a C++ enum.** Namespace: `SeinARTS.CommandType.*`. Reasons: designers can add their own command types in content without C++ changes; replay format is version-stable (tag strings, not enum ordinals); pairs naturally with the existing `SeinARTS.CommandContext.*` namespace.
- **Command payload: common fields + optional typed payload.** `FSeinCommand` carries a fixed set of common fields (PlayerID, CommandType tag, IssueTick, EntityHandle, TargetEntity, TargetLocation, ContextOrAbilityTag) plus an **optional `FInstancedStruct Payload`** for command types that need more. Payload types are named USTRUCTs (`FSeinShiftQueuePayload`, `FSeinBrokerOrderPayload`, etc.), never free-form bags. Simple commands leave the payload empty.
- **Observer commands are non-sim but logged.** Camera, selection, and control-group assignments enter the txn log so replays can reconstruct player-POV UI state. They do NOT mutate sim state. `FSeinCommand::IsObserverCommand()` (already exists) is the partition. Communication commands (pings, chat, emotes / quick-chat) are also logged — they convey player intent / info even though they don't mutate sim state. UI state beyond selection + camera (tooltips, minimap zoom, build-menu-open, cursor position) is NOT tracked.
- **Observer commands never influence the sim.** If a designer wants an "observer with influence" pattern (director cam emitting commands), they make a custom player controller that issues normal sim commands through the lockstep stream. Observer = no sim influence is a hard rule.
- **One flat command log.** Sim and observer entries interleaved, tick-ordered. Competitive-replay stripping is deferrable; if it becomes a need, we add a pass that filters observer entries post-facto.
- **Rejection UX via visual events.** When the sim rejects a command (entity died between click and processing, cost deficit, on cooldown, out of range, activation-blocked tag), it emits a `FSeinVisualEvent::MakeCommandRejectedEvent(reason)` for the UI to surface ("not enough munitions", "can't do that now"). Pure UX addition — the sim already rejects silently; this makes failures discoverable.
- **Client-side validation is additive, not authoritative.** The player controller MAY pre-validate commands before emitting to save a round-trip for obviously-invalid inputs. But it never short-circuits sim validation — state at command-emit time can differ from state at command-processing time by several ticks in lockstep. Sim is always the authority.
- **Smart command resolution happens sim-side.** Right-click on the ground vs an enemy vs a resource vs a friendly transport resolves to different abilities. The resolver lives in the CommandBroker (§4) — it inspects the target, queries member capability maps, and dispatches per-member ActivateAbility calls. The player

controller emits one pre-resolution command regardless of target type.

- **AI uses the same lockstep buffer as human players.** No separate AI channel. AI is a server-side command issuer; its commands flow through the same `FSeinCommand` stream. Makes replay / multiplayer-in-lobby / AI-vs-AI all reproducible from the command log alone.
- **`FSeinPlayerID` alone attributes commands.** AI, scripted events, and replay-playback register as "players" with their own IDs. No secondary `CommandSource` enum. If debugging surfaces a need, add later.

Non-goals

- Separate sim / observer txn-log streams (one flat log; filter on export if needed).
- Compile-time command dispatch via C++ enums (blocks designer extensibility).
- Sim-state-reading pre-validation on the client (lockstep drift makes it unsafe as the authority).
- Per-cursor-position replay (tracked state bloat with low value).
- Per-command source attribution (`Human/AI/Script`) beyond `FSeinPlayerID`.

Implementation deltas that fall out

- Convert `ESeinCommandType` enum to an `FGameplayTag CommandType` field. Framework-shipped types live in `SeinARTS.CommandType.*` (ActivateAbility, CancelAbility, QueueProduction, CancelProduction, SetRallyPoint, Ping, CameraUpdate, SelectionChanged, ChatMessage, Emote, etc.).
- Add optional `FInstancedStruct Payload` field to `FSeinCommand`.
- Add `FSeinVisualEvent::MakeCommandRejectedEvent(reason, context)` + emit from all `ProcessCommands` rejection branches.
- Retire `IsObserverCommand()` as an enum-switch; either add an `FGameplayTag` flag on the tag (`SeinARTS.CommandType.Observer.*` prefix) or a sidecar set of observer-type tags — design choice during implementation.
- Extend the observer command set: `ChatMessage`, `Emote`, `Ping` already exist; add `ControlGroupAssigned` explicitly.

---

4. CommandBrokers `[locked]`

Sim-side primitive that mediates between a player's (or AI's) single dispatch-level command and the dynamic set of entity members receiving it. Replaces the old "formation manager" concept from the pre-ECS plugin, generalized.

Purpose

Player right-clicks with 20 heterogeneous units selected (sniper + 2 rifle squads + wheeled vehicle + 2 tanks). Naïve dispatch (every unit gets the same move order to the same point) produces chaos. The CommandBroker:

1. Wraps the set of units as a single sim entity with one-to-many membership
2. Polls members' ability components to build an aggregated capability map
3. Runs a designer-pluggable resolver that decides **which members execute what ability at which location**
4. Dispatches per-member `ActivateAbility` calls internally (sim-side, NOT logged to txn — they're deterministic consequences of the one broker order)
5. Tracks all-members-done / any-member-done completion and propagates back
6. Holds the command queue for shift-chained orders
7. Self-spawns when an order arrives; self-culls when empty

The txn log carries ONE entry per player click regardless of selection size (`FSeinCommand { CommandType=SeinARTS.CommandType.BrokerOrder, MemberHandles, TargetLocation, Context }`). Each

client's sim spawns or reuses a broker, runs the resolver, dispatches.

Decisions

- **Entity, not loose UObject.** CommandBrokers live in the entity pool with a `FSeinEntityHandle`. Implicit `FSeinTagData` (per §1). A dedicated `FSeinCommandBrokerData` component carries membership, resolver, anchor, centroid, capability map, queue, active order state.
- **One broker per member, strictly.** The old invariant holds. Adding a member to a new broker first evicts it from its previous broker. Old broker auto-culls if empty. Bidirectional reference: each entity's component storage gets a `FSeinBrokerMembershipData { FSeinEntityHandle CurrentBrokerHandle }` that the broker's add/remove ops keep in sync.
- **Broker ownership is wholly-single-player (invariant).** A broker's member set must all be owned by the same `FSeinPlayerID` (the broker's owner, inherited from its members at spawn). The broker primitive enforces this at instantiation — if the incoming member list contains entities owned by different players, the broker either auto-filters to the issuing player's entities or rejects outright, depending on the dispatch path. Caller-side filtering (player controller checking ownership before emitting the broker order) is a convenience; broker-side enforcement is the trust boundary. If post-filter member set is empty, no broker is created and no command fires. **Allied-command sharing** (in some match modes, allies can command each other's units) is a match-settings-controlled loosening of the "owner scope" used by dispatch — the primitive invariant is unchanged; match settings widen the set of players whose entities count as "ownable" for dispatch purposes.
- **Command-driven creation, not selection-driven.** Selecting units creates nothing sim-side. Dispatching an order from a selection creates/reuses a broker. Matches the old behavior and keeps selection a pure client-side concept.
- **Wraps even size-1 selections.** Single-unit orders go through the broker pathway identically. Cost is minimal (one entity + one component); benefit is one code path for command dispatch, completion tracking, UI binding, and queue management.
- **Capability map is polled per-dispatch, rebuilt on membership change.** The broker inspects each member's `FSeinAbilityData::AbilityInstances` to build `TMap<FGameplayTag, TArray<FSeinEntityHandle>>` — "which members can service each ability tag." Cached on the component, invalidated when members are added/removed. Polled fresh on each command dispatch for correctness. This enables heterogeneous dispatch: Attack order → only Attack-capable members; other members fall through to a designer-defined default (e.g., Move-toward-target for non-combatants).
- **Resolver is a designer-subclassable UObject.** `USeinCommandBrokerResolver` (abstract, Blueprintable) with `ResolveDispatch(members, contextTag, targetLocation, targetEntity) → dispatch plan` and `ResolvePositions(members, anchor, facing) → per-member positions`. Default implementation in C++. Designers subclass in BP or C++ for custom behavior (tight-rank resolver, class-clustered spacing, formation-shape presets, etc.). Resolver output integrates with the nav composition (§12) — per-member target positions feed into flow-field-driven steering via each unit's `USeinMovementProfile`.
- **Resolver class selection via plugin settings.** `USeinARTSCoreSettings` gets a `TSoftClassPtr<USeinCommandBrokerResolver> DefaultResolverClass` field. Framework ships a sane default (class-clustered uniform spacing). Designers can override globally via settings or per-match by setting a different default before `StartSimulation()`.
- **Transform: centroid + anchor, both fields.**
  - `FFixedVector Centroid` — dynamically updated from member positions each tick (or on-demand); used for UI centering, camera focus, distance queries
  - `FFixedVector Anchor` + `FFixedRotator AnchorFacing` — set when a broker order dispatches; "where the formation is trying to stand"; used by tight-formation resolvers and visual aids (banner markers, marching-to indicators)
- **Queue lives on the broker, not on members.** Shift-click chains orders; the broker's

`TArray<FSeinBrokerQueuedOrder> OrderQueue` is the sole source of truth for "this group's ongoing plan." Members only ever execute one active ability at a time (dispatched from the current broker order). When a member dies, the broker can redistribute or continue without them — member-loss doesn't lose queued work.

- **No broker-level abilities.** The broker does NOT have a designer-authored `FSeinAbilityData`. It has dispatch logic (in the resolver) and queue logic (in its component), but no USeinAbility instances of its own. Abilities belong to members.
- **Control groups are not brokers.** Control groups are a client-side selection preset (the ControlGroup = cached array of entity handles the player bound to a hotkey). Pressing `1` selects those units client-side; issuing an order then creates/reuses a broker just like any other order. Control-group membership and broker membership are orthogonal. A unit in ControlGroup 1 and ControlGroup 2 (both client-side sets) will be in whichever broker its last-dispatched order created, regardless of control group.
- **Completion semantics: all-members-done by default.** The active order is "complete" when every member's executing ability ends (via OnEnd, cancelled or otherwise). Resolver can override with any-member-done or any-other custom predicate. On completion, the broker pops its next queued order or self-culls if queue is empty.

Non-goals

- Selection-driven broker creation (selection is client-side, brokers are command-driven).
- Nested / overlapping brokers (one broker per member, strict).
- Per-member order queues (queue lives on the broker).
- Broker-level ability kits (dispatch happens through the resolver; abilities belong to members).
- Cross-broker coordination primitives (if a game needs that, it's a higher-level scripted concept, not a framework primitive).

Implementation deltas that fall out

- New entity archetype: `USeinCommandBrokerArchetype` (or just a convention — spawns into entity pool without a backing Blueprint class since brokers have no render presence).
- New component: `FSeinCommandBrokerData { TArray<FSeinEntityHandle> Members; TObjectPtr<USeinCommandBrokerResolver> Resolver; FFixedVector Centroid; FFixedVector Anchor; FFixedRotator AnchorFacing; TMap<FGameplayTag, TArray<FSeinEntityHandle>> CapabilityMap; TArray<FSeinBrokerQueuedOrder> OrderQueue; bool bIsExecuting; FGameplayTag CurrentOrderTag; }`.
- New component: `FSeinBrokerMembershipData { FSeinEntityHandle CurrentBrokerHandle }` — added to entities when they join a broker, drives eviction-on-add.
- New UObject: `USeinCommandBrokerResolver` with virtual / BlueprintNativeEvent dispatch + positions methods.
- New tick system: `FSeinCommandBrokerSystem` (AbilityExecution or PostTick) that updates centroids, handles completion callbacks, drives queue advancement, culls empty brokers.
- New BPFL: `USeinCommandBrokerBPFL` for BP-side introspection (get members, get centroid, get active order, etc.).
- `FSeinCommand` gains a `BrokerOrder` CommandType tag; `ProcessCommands` handler creates/reuses a broker from the payload's member list, then invokes the resolver.
- `ProcessDeferredDestroys` on a member entity must remove from the member's current broker; if broker becomes empty, cull it.
- Plugin settings: `TSoftClassPtr<USeinCommandBrokerResolver> DefaultResolverClass` + framework-default resolver class implementation.

---

5. Resources `[locked]`

Player-level economy. Everything that "costs" something (abilities, production, upkeep) bills against this layer.

Decisions

- **Stockpile semantics.** Resources are integer-like `FFixedPoint` totals. Income ticks add to the stockpile; costs deduct at spend time. Matches standard stockpile-based RTS economies (the majority of the genre). A continuous-flow model (rate-based economies with buffer caps and stall semantics, as seen in some massive-scale RTS games) is an explicit non-goal — a game that wants flow can approximate it by ticking small stockpile deltas, but the framework does not implement continuous-flow economy math.
- **Tag-keyed resources.** Resources are identified by `FGameplayTag` in the `SeinARTS.Resource.*` namespace (designer-extensible). `FSeinPlayerState::Resources` becomes `TMap<FGameplayTag, FFixedPoint>`. All cost declarations (ability cost, production cost) become tag-keyed. Consistent with CommandType / CommandContext / tech / unit-class tagging across the framework; edit-time tag-picker replaces free-text FName authoring.
- **Resource catalog in plugin settings + per-faction kits (hybrid).** Two layers:
  - `USeinARTSCoreSettings::ResourceCatalog` — a project-wide array of `FSeinResourceDefinition` entries. Each entry declares a resource's tag, display name, icon, cap, overflow behavior, spend behavior, and default starting value.
  - `USeinFaction::ResourceKit` — a per-faction list of "which catalog resources this faction uses," optionally with overrides (starting value, cap, etc.) and faction-specific additions (e.g., a faction with a unique resource only it uses).
  - Universal resources (typical examples: manpower, munitions, fuel) are defined once in the catalog and every faction kit references the same entries. No duplication.
- **Cap / overflow / spend behavior is per-resource, designer-declared, with sane defaults.**
  - Default overflow behavior: **clamp at cap** (income wasted beyond cap).
  - Default spend behavior: **reject on insufficient** (hard deny).
  - Designers may override per-resource: allow-debt (for upkeep-driven economies), overflow-into-score (ranked scoring modes), or custom via a resolver hook.
- **Income sources — all four supported from day one.**
  - **Passive / tick income** — a flat per-tick delta configured on the faction kit's resource entry.
  - **Structure / entity income** — entities carry `FSeinResourceIncomeData` contributing to their owner's income rate. Capture points, refineries, extractors.
  - **Ability-granted** — abilities may deposit one-time resource injections via a BPFL. Scavenge / loot-drop patterns, energy-surge abilities, research-completion bonuses.
  - **Trade / gift** — cross-player transfer commands. Guarded by match settings (see Sharing below).
- **Upkeep / drain — all three supported.**
  - **Flat upkeep drain.** Entities declare a per-tick resource drain on their owner. Drain deducts during the Resource tick.
  - **Pop / supply cap** — a soft resource. Units contribute to pop consumption; spending that would exceed the cap is rejected. Pop is just a resource with designer-configured spend/overflow behavior (no distinct primitive).
  - **Income-rate modifiers.** Army-size-scaled upkeep (more units → lower income rate) is expressed as a modifier on the income-rate attribute, not a flat drain. Army size triggers archetype modifiers that reduce income rate; existing modifier system handles the math. No new primitive.
- **Resource sharing is match-settings-controlled, not a plugin-wide setting.**
  - Default: strictly per-player.

  - Match settings allow enabling team-shared pools (co-op classic), designer-hybrid sharing (some resources shared, others private), or player-gift commands (explicit transfer only).
  - Implies a `FSeinMatchSettings` concept — flagged for future design pass (see §16 or a new match-settings section TBD).

- **Unified cost model.** `FSeinResourceCost` is a single struct type used for both ability cost and production cost. Single validation function (`CanAfford`), single deduction function (`Deduct`), single refund function (`Refund`). Cost semantics:

```
USTRUCT(BlueprintType)
struct FSeinResourceCost
{
    TMap<FGameplayTag, FFixedPoint> Amounts;
    // Future: modifiers (cost reductions from tech), time cost for production
};
```

- **Cost check is a hardcoded validation, not a tag query.** `CanAfford(Player, Cost)` iterates the cost map and compares against player state. Simple, fast, deterministic. Tag-based "requires at least X munitions" expressed as ActivationBlockedTags would require tag-enumerations at thresholds (awkward) — hardcoded path wins.
- **Symmetric availability API.** Both abilities and production expose an availability query (affordable? on-cooldown? blocked? prerequisite-met?) returning a struct. UI binds uniformly to ability availability and production availability.
  - `USeinAbilityBPFL::SeinGetAbilityAvailability(entity, abilityTag) → FSeinAbilityAvailability`
  - `USeinProductionBPFL::SeinGetProductionAvailability(entity, archetypeTag) → FSeinProductionAvailability` (exists today; reconcile shape with the ability version).

Non-goals

- Continuous-flow economy semantics (rate-based with buffer caps and stall math). Out of scope; workaround via small stockpile deltas.
- Plugin-wide resource-sharing setting. Must be match-level.
- Pop cap as a distinct primitive. It's a resource with designer-configured cap/spend behavior.
- Per-ability custom cost resolvers. `FSeinResourceCost` is flat; modifier effects (tech discounts) hook into resolution via the existing modifier system, not via custom cost code.

Implementation deltas that fall out

- Replace `TMap<FName, FFixedPoint>` with `TMap<FGameplayTag, FFixedPoint>` at all call sites: `FSeinPlayerState::Resources`, `USeinAbility::ResourceCost`, `FSeinProductionQueueEntry::Cost`, relevant BPFL signatures, visual event payloads.
- Add `FSeinResourceDefinition` struct (tag, display name, icon, cap, overflow behavior enum, spend behavior enum, default starting value).
- Add `USeinARTSCoreSettings::ResourceCatalog: TArray<FSeinResourceDefinition>`.
- Add `USeinFaction::ResourceKit: TArray<FSeinFactionResourceEntry>` (references a catalog entry by tag + optional overrides).
- Add `FSeinResourceCost` struct; refactor ability cost and production cost to share it.
- Add `USeinResourceBPFL` with `CanAfford`, `Deduct`, `Refund`, `GrantIncome`, `Transfer` (transfer respects match-settings sharing rules).
- Wire `USeinAbility::ActivateAbility` to check affordability + deduct cost; wire `DeactivateAbility(bCancelled=true)` to refund if the ability's `bRefundOnCancel` flag is set (flag decision pending §6 Abilities Q&A).
- Wire income computation: a `FSeinResourceSystem` (exists) ticks per-player income, combining faction

passive + entity-source income + modifier effects (for army-size-scaled upkeep).
- Flag: **match settings** are a future topic. Resource-sharing mode, starting-value overrides, and per-resource match-level tweaks live there. Create a match settings primitive in a later pass.

---

6. Abilities `[locked]`

The unit's implementation of commands. Activated explicitly or passively, with tag-based arbitration, latent-action execution, BP-scriptable lifecycle, and declarative validation.

Decisions

- **Tag-based arbitration** (recap). Three BP-editable containers on `USeinAbility`:
  - `ActivationBlockedTags` — entity tags that refuse this ability from activating. Combined with `OwnedTags` on the same ability to self-block (grenade-during-channel).
  - `OwnedTags` — tags this ability grants to the entity while active. Applied via refcount grant (per §2); removed via refcount ungrant on deactivate.
  - `CancelAbilitiesWithTag` — on activate, cancels any active ability (including self) whose `OwnedTags` intersect this set. An ability listing one of its own owned tags here gets self-cancelling reissue (repeat-right-click Move).
- **Activation flow in `ProcessCommands`**:
  1. Cooldown check (reject if on cooldown).
  2. `ActivationBlockedTags` vs entity tags (reject if blocked).
  3. Declarative target validation (range, valid-target-tags, LOS — see below).
  4. `CanActivate` BP event (returns false → reject). Escape hatch for non-declarative rules.
  5. Affordability check via `FSeinResourceCost` (reject if unaffordable).
  6. Deduct cost.
  7. Cancel any active abilities whose `OwnedTags` match `CancelAbilitiesWithTag`.
  8. Start cooldown (if `CooldownStartTiming == OnActivate`).
  9. Apply `OwnedTags` via refcount grant.
  10. Invoke `OnActivate` BP event.
- **Cost deducts on activation success (Q1a).** Mirrors production cost semantics. On cancel, cost refunds if `bRefundCostOnCancel` is true.
- **`bRefundCostOnCancel` — default true (Q2a).** Matches typical RTS economy: cancelling is "didn't really happen" from the ledger's perspective. Designers set false for punitive-cancel abilities. `OnEnd(bWasCancelled=true)` handler runs *after* the framework refund — designers can still apply partial refunds or extra cleanup in BP.
- **`bRefundCooldownOnCancel` — default false (Q3c).** Cooldowns represent "you used this recently." Cancelling partway still counts as using it. Designers opt in to true for abilities where pre-commit cancel should be free (typically paired with `CooldownStartTiming == OnEnd` so the cooldown hadn't even started yet).
- **`CooldownStartTiming` — enum, per-ability (Q4c).**

  ```
  enum class ESeinCooldownStartTiming : uint8
  {
      OnActivate,  // Cooldown begins immediately on successful activation (sprint buffs, mos
      OnEnd,       // Cooldown begins when the ability ends (grenade throw — cooldown starts
  };
  ```

  Default `OnActivate`. `OnEnd` for long-cast / "the thing really finishes after some time" abilities.

- **Target validation is hybrid declarative + BP (Q5c + Q6 hybrid).** Declarative fields on the ability class cover common cases:
  - `FFixedPoint MaxRange` (0 = unlimited) — distance from owner entity to target location/entity
  - `FGameplayTagQuery ValidTargetTags` — tag query the target entity must satisfy (e.g., "has `Unit.Hostile`")
  - `bool bRequiresLineOfSight` — integrates with §11 Vision
  - `ESeinAbilityTargetType TargetType` — None / Location / Entity / Both (exists today)

  `CanActivate` BlueprintNativeEvent remains the escape hatch for unusual rules (facing constraints, high-ground requirements, combo-window checks). Sim runs declarative validation first, then calls `CanActivate` only if declarative passes.

- **Out-of-range behavior — enum, per-ability (Q7).**

  ```
  enum class ESeinOutOfRangeBehavior : uint8
  {
      Reject,        // Ability fails if out of range (grenade, snipe)
      AutoMoveThen,  // Framework auto-queues a move-to-range prefix; ability re-attempts on
  };
  ```

  Default `Reject`. `AutoMoveThen` is the classic "right-click enemy → unit moves into range then attacks" pattern common in attack-order systems. Implementation note: `AutoMoveThen` needs CommandBroker integration — the broker prepends an internal Move dispatch before the ability dispatch. Details defer to implementation pass; design-locked here.

- **Passives are actives that auto-activate at spawn (Q8a).**
  - `bIsPassive = true` triggers auto-activation during `InitializeEntityAbilities`. Otherwise lifecycle is identical to active abilities.
  - Passives participate fully in tag arbitration — they can be cancelled by other abilities' `CancelAbilitiesWithTag`.
  - **Cancelled passives do NOT auto-reactivate.** If a passive is cancelled mid-match (damage cancels stealth, another ability's `CancelAbilitiesWithTag` hits), it stays cancelled until a BPFL re-activates it or the entity respawns. Designers write explicit reactivation hooks for narrow cases (stealth re-engages after combat ends).
  - Passives have no cost and no cooldown semantics. If you want toggleable-with-upkeep behavior, model it as a normal active ability with indefinite duration + upkeep via §5 Resources.

- **DefaultCommands move from archetype def to `USeinAbilitiesComponent` (Q9).** `FSeinAbilityData` gains `TArray<FSeinCommandMapping> DefaultCommands` + `FGameplayTag FallbackAbilityTag`. The `ResolveCommand` method moves with the data. Archetype def no longer carries command-mapping concerns.
  - **Sole source of truth.** No ability-self-declaration hybrid. Designers author all command→ability mappings on the abilities component. Single view, single edit surface.

- **Ability upgrades use existing primitives, not a new one (Q10).** Three patterns, all expressible today:
  - **Stat upgrades** (movespeed +30%, damage +50%) → `FSeinModifier` effects scoped to archetype or instance (exists via §7 Effects + `FSeinPlayerState::ArchetypeModifiers`).
  - **Ability replacement** (basic Move → Move-with-Sprint) → grant new ability class, revoke old. Command resolver picks the new tag. Typically driven by a tech completion or effect application.
  - **Ability enhancement** (Move gains a charge ability) → grant additional ability; command resolver's priority picks appropriately based on context.
  - No "upgrade" primitive; upgrades are compositions of grant/revoke/modify.

- **BP authors wire `End Ability` on terminal pins** (convention, recap). The framework's re-issue safety net (`CancelAbilitiesWithTag` resolving to self-cancel via `OwnedTags`) covers most cases where an author forgets. But un-re-issued abilities stuck active still block passives, UI states, etc. Documentation reinforces this in CLAUDE.md's Pitfalls section.

Non-goals

- Auto-revive hook for cancelled passives. Keep the model simple; designers write explicit reactivation logic for the narrow cases that want it.
- Multi-phase cost (activate-then-commit GAS-style). Flat deduct on activation success is sufficient.
- Interrupt/abort/fail as distinct lifecycle states. The `bWasCancelled` flag on `OnEnd` covers the meaningful distinction; why-cancelled semantics can be layered into BP if a designer needs it.
- Ability "upgrade" as a first-class primitive. Compose via effects + grant/revoke.
- Sim-cached range resolution (beyond a per-activate check). If a designer wants continuous range-tracking (e.g., auto-disable when target moves out of range), they implement it in `OnTick` or via effects.

Implementation deltas that fall out

- **`USeinAbility` header:**
  - Add `bool bRefundCostOnCancel = true;`
  - Add `bool bRefundCooldownOnCancel = false;`
  - Add `ESeinCooldownStartTiming CooldownStartTiming = ESeinCooldownStartTiming::OnActivate;`
  - Add `ESeinOutOfRangeBehavior OutOfRangeBehavior = ESeinOutOfRangeBehavior::Reject;`
  - Add `FFixedPoint MaxRange = FFixedPoint::Zero; // 0 = unlimited`
  - Add `FGameplayTagQuery ValidTargetTags;`
  - Add `bool bRequiresLineOfSight = false;`
  - Remove `AppliedOwnedTags` runtime field (refcount replaces diff-on-activate, per §2 delta).
- **`USeinAbility::ActivateAbility`:** integrate cost-deduct via `USeinResourceBPFL::Deduct` with the resolved `FSeinResourceCost`. Apply `OwnedTags` via refcount grant (no more diff). Start cooldown per `CooldownStartTiming`.
- **`USeinAbility::DeactivateAbility`:** on `bCancelled=true`, refund cost if `bRefundCostOnCancel`, reset cooldown if `bRefundCooldownOnCancel`. On any end, start cooldown if `CooldownStartTiming == OnEnd` and not already refunded. Refcount ungrant of `OwnedTags`.
- **`USeinAbility::ResourceCost`:** refactor to `FSeinResourceCost` (per §5) — `TMap<FGameplayTag, FFixedPoint>` instead of `TMap<FName, FFixedPoint>`.
- **`ProcessCommands::ActivateAbility`:** implement the full activation flow above, including declarative validation before `CanActivate`.
- **Declarative target validation:** add helper `FSeinAbilityValidation::ValidateTarget(Ability, Owner, Target, Location)` that checks range + tags + LOS.
- **`AutoMoveThen`:** integrate with CommandBroker — when an ability command arrives with this behavior and target is out of range, the broker prepends an internal Move order to get within range, then re-attempts the ability on arrival. Implementation details TBD.
- **DefaultCommands migration:**
  - Move `DefaultCommands`, `FallbackAbilityTag`, and `ResolveCommand` from `USeinArchetypeDefinition` to `FSeinAbilityData`.
  - Update `SeinPlayerController.cpp:1000, :1120` call sites to resolve via the abilities component.
  - Archetype def shrinks (identity + cost + tech + `BaseTags` + `GrantedModifiers` only).
- **Availability API:** add `USeinAbilityBPFL::SeinGetAbilityAvailability(entity, abilityTag) → FSeinAbilityAvailability` returning an aggregate struct (affordable / on-cooldown / blocked-by-tag / out-of-range / custom-CanActivate-failed). UI binds uniformly. Matches `USeinProductionBPFL::SeinGetProductionAvailability` in shape.

---

7. Effects `[locked]`

Runtime modifiers applied to entities or to a player's archetype-scope state. Health regen, damage-over-time, stat buffs, production discounts, veterancy stacks, cover bonuses, tech-granted upgrades — all expressed as effects.

Decisions

- **Single primitive.** One `FSeinActiveEffect` struct handles instant, finite-duration, infinite, and periodic effects. No split between "one-shot" and "persistent" — duration and tick-interval are axes, not types.
- **Duration model.**
  - `Duration == FFixedPoint::Zero` → instant (applies, fires hooks, removes same-tick)
  - `Duration > Zero` → finite (decrements each PreTick, removed when exhausted)
  - `Duration < Zero` (sentinel `-1`) → infinite (never auto-expires; removed only by explicit API or matching `RemoveEffectsWithTag`)
- **Periodic payload.** Optional `TickInterval` field on the effect. `Zero` → no periodic callback. `> Zero` → fires `OnTick` hook every N ticks while active. Orthogonal to duration: an effect can have `Duration=30, TickInterval=1` (DoT for 30 ticks, 1 damage per tick) or `Duration=-1, TickInterval=5` (passive aura that fires every 5 ticks forever).
- **Stacking — per-effect rule, default Stack (Q1d).**

  ```
  enum class ESeinEffectStackingRule : uint8
  {
      Stack,        // default: CurrentStacks++; modifiers multiply by stacks; duration refre
      Refresh,      // CurrentStacks stays at 1; duration refreshes
      Independent,  // add a separate instance; each tracks its own timer
  };
  ```

  `Stack` is the default because it covers the most common case (DoTs, buffs) cleanly.
- **Stacking cap: `MaxStacks` integer (Q2b).** `MaxStacks = 1` means non-stacking. Higher values gate. No dedicated "unlimited" sentinel — designers use a big number (`INT32_MAX` or a project convention like `9999`) if they want effectively-unlimited.
- **Source-death behavior — `bRemoveOnSourceDeath` flag, default false (Q4c).** Most effects persist past source death (bleed from dead attacker, tech modifier after researcher-building dies). Opt in for source-dependent effects (toggle auras from a living commander, active-maintenance buffs).
- **Conditional effects are emergent, not a primitive (Q3).** The framework does NOT support "auto-remove when condition X becomes false." Instead, the condition source drives apply/remove explicitly — a cover zone has its own passive ability / polling system that applies cover effects to entities entering and removes them on exit. Keeps the effect primitive dumb and the condition logic localized. Matches the past plugin's approach.
- **Rich effect definitions (Q6).** Beyond modifiers, an effect can:
  - **Grant tags** (`GrantedTags` container) — applied via refcount grant on apply, ungrant on remove. Same tag refcount system as abilities' `OwnedTags` (§2).
  - **Remove other effects** (`RemoveEffectsWithTag` container) — on apply, strips any existing effect whose `EffectTag` matches. Anti-poison removes Poison.
  - **Fire BP hooks** — `OnApply`, `OnTick`, `OnExpire`, `OnRemoved` BlueprintImplementableEvents on the effect class for custom logic.
- **Effect authoring via UObject subclasses (Q7b).** `USeinEffect : UObject` is a Blueprintable base class; designers subclass it in BP or C++. Mirrors `USeinAbility` / `USeinActorComponent` authoring: the UObject carries the class-level config (modifiers, granted tags, duration defaults, stacking rule, BP event graph), while `FSeinActiveEffect` in component storage is a lightweight sim-side struct holding a class reference + per-instance runtime state (CurrentStacks, RemainingDuration, Source, EffectInstanceID).
  - CDO-config, instance-runtime-state split. No per-instance config override in the initial design; add

`FSeinEffectOverride` as a follow-up if a real use case surfaces ("Poison with 3s duration" vs "Poison with 10s duration" using the same class).

- **Application — three initiation paths (Q8).**
  1. **Ability-applied.** A BP ability calls `USeinEffectBPFL::SeinApplyEffect(target, class, source)`. The primary path.
  2. **Condition-source-applied.** A cover zone or aura entity's polling logic applies/removes effects as entities enter/leave range. Uses the same BPFL — just called from a different context.
  3. **Effect-from-effect.** An effect's `OnApply`/`OnTick` BP graph can apply another effect. Recursion handling below.
- **Recursion handling via apply-batching (Q9c).** Effects applied *during* a tick's `OnApply` / `OnTick` / `OnExpire` / `OnRemoved` hook go into a pending-apply queue. The queue is drained at the *next* PreTick phase. An effect cannot apply another effect that fires hooks within the same tick. Prevents infinite-apply loops naturally. Side benefit: designers thinking in terms of "this tick's effect cascade" see deterministic ordering.
- **Dev-mode apply-count warning.** If an entity's active effect count exceeds a threshold (`256` default, configurable in plugin settings), log a warning once per threshold crossing. Zero runtime cost in shipping; catches runaway feedback loops at design time. Not a gate — effects still apply normally.
- **Three scopes, one primitive.** An effect is applied at one of three scopes, determined by its `Scope` field. Same primitive, same lifecycle, same stacking, same tags, same hooks — different storage location and modifier-target semantics.
  - **`Instance`** — effect lives in the target entity's `FSeinActiveEffectsData`. Modifiers target entity component fields (health, damage, movespeed, etc.). Example: a single squad gets a rifle upgrade; veterancy stacks on one unit; DoT bleeding on a specific entity.
  - **`Archetype`** — effect lives in the player state's `ArchetypeEffects` list. Modifiers target entity component fields matched by `TargetArchetypeTag` at attribute-resolve time; every entity owned by the player with the matching tag receives the influence. Example: "all Infantry +20% damage" (one effect with Archetype scope, modifiers using `TargetArchetypeTag = Unit.Infantry`). Empty/wildcard archetype tag matches every owned entity ("all units of this player +10% speed").
  - **`Player`** — effect lives in the player state's `PlayerEffects` list. Modifiers target player-state-level fields directly (resource income rates, resource caps, pop cap, upkeep rates). Example: "+10% Manpower income rate", "+50 max population", "-20% Fuel upkeep."
- **`ResolveAttribute` is scope-aware.** Entity attribute queries iterate active instance-scope effects on the entity + archetype-scope effects on the owner whose `TargetArchetypeTag` matches the entity. Player-state attribute queries (income rates, caps) iterate the player's player-scope effects. The same modifier primitive applies at each level; the resolver just walks the right list for the attribute being queried.

Non-goals

- "Conditional effect" as a framework primitive. Condition sources drive apply/remove themselves.
- Separate "periodic effect" primitive. `TickInterval` field on the one primitive covers it.
- Per-instance config override at apply time (Duration, MaxStacks, etc.). Follow-up work if surfaced.
- Runtime cycle-detection in effect-from-effect chains. Per-tick batching + dev-mode warning is sufficient.
- Unlimited stacks sentinel. Designers use a large number.
- Refresh-on-duplicate-apply as the default. Stacking is the default; designers pick Refresh per effect.

Primitive shapes

```
USTRUCT(BlueprintType)
struct FSeinActiveEffect
{
    TSubclassOf<USeinEffect> EffectClass;  // class reference; CDO holds the config
```

```
    // Runtime state (per-instance)
    uint32 EffectInstanceID;
    int32 CurrentStacks;
    FFixedPoint RemainingDuration;         // ticks until expire; -1 for infinite
    FFixedPoint NextTickTime;               // ticks until next OnTick fire
    FSeinEntityHandle Source;               // may be invalidated; check with EntityPool
};

UCLASS(Blueprintable, Abstract)
class USeinEffect : public UObject
{
    // --- Configuration (BP defaults) ---
    UPROPERTY(EditDefaultsOnly) FGameplayTag EffectTag;
    UPROPERTY(EditDefaultsOnly) TArray<FSeinModifier> Modifiers;
    UPROPERTY(EditDefaultsOnly) FGameplayTagContainer GrantedTags;
    UPROPERTY(EditDefaultsOnly) FGameplayTagContainer RemoveEffectsWithTag;
    UPROPERTY(EditDefaultsOnly) FFixedPoint Duration;             // 0=instant, -1=infinite, >0
    UPROPERTY(EditDefaultsOnly) FFixedPoint TickInterval;          // 0=none, >0=periodic
    UPROPERTY(EditDefaultsOnly) int32 MaxStacks = 1;
    UPROPERTY(EditDefaultsOnly) ESeinEffectStackingRule StackingRule = ESeinEffectStackingRule:
    UPROPERTY(EditDefaultsOnly) bool bRemoveOnSourceDeath = false;
    UPROPERTY(EditDefaultsOnly) ESeinModifierScope Scope = ESeinModifierScope::Instance;
    // ^ ESeinModifierScope: { Instance, Archetype, Player } — determines storage location and

    // --- BP hooks ---
    UFUNCTION(BlueprintImplementableEvent) void OnApply(FSeinEntityHandle Target, FSeinEntityHa
    UFUNCTION(BlueprintImplementableEvent) void OnTick(FSeinEntityHandle Target, FFixedPoint De
    UFUNCTION(BlueprintImplementableEvent) void OnExpire(FSeinEntityHandle Target);
    UFUNCTION(BlueprintImplementableEvent) void OnRemoved(FSeinEntityHandle Target, bool bByExp
};
```

Implementation deltas that fall out

- Promote `FSeinActiveEffect.Definition` (currently `FSeinEffectDefinition` struct) into the `USeinEffect` UObject class. Instances carry a `TSubclassOf<USeinEffect>` reference; config reads go through the CDO.
- Add `USeinEffectBPFL::SeinApplyEffect(target, class, source) → FSeinEffectHandle` (or effect instance ID) so BP scripts can apply and track effects.
- Add `USeinEffectBPFL::SeinRemoveEffect(target, instanceID)` and `SeinRemoveEffectsWithTag(target, tag)`.
- Wire `GrantedTags` into the refcount grant/ungrant path (per §2) so effects and abilities share the same tag plumbing.
- Refactor `FSeinEffectTickSystem` to drive both duration decrement and `TickInterval` firing. Pending-apply queue drained at PreTick.
- Dev-mode warning system: log once per entity when active effect count crosses the threshold (configurable in plugin settings).
- Extend `ESeinModifierScope` enum with `Player` value (currently only `Instance` and `Archetype`).
- Add two player-scoped effect lists on `FSeinPlayerState`: `TArray<FSeinActiveEffect> ArchetypeEffects` (Archetype scope) + `TArray<FSeinActiveEffect> PlayerEffects` (Player scope). Alongside the existing flat `ArchetypeModifiers` during migration.
- Extend `FSeinModifier::TargetComponentType` semantics to accept player-state struct types (resource-rate sub-structs on `FSeinPlayerState`) in addition to entity component structs. Resolver routes to the right sim location based on the modifier's parent effect scope.
- `ResolveAttribute` splits into two codepaths: entity-attribute resolve (iterates Instance + matching-tag Archetype) and player-state-attribute resolve (iterates Player).
- `USeinEffect` class integrates with editor: thumbnail renderer, asset type actions, factory. Follows the pattern established for `USeinAbility` / `USeinActorComponent` / widget blueprints.

---

8. Production `[locked]`

Unit production, research production, upgrade production. Builds on the Resources primitive (§5) and uses the same `FSeinResourceCost` as abilities (§6).

Decisions

- **Cost deduction timing is per-resource, not per-item (Q1, Q6).** Declared on the resource's catalog entry (`FSeinResourceDefinition::ProductionDeductionTiming`). Two timings supported:
  - **`AtEnqueue`** — cost commits when the item hits the queue. Prevents over-queue. Default for most stockpile resources (minerals, munitions, fuel).
  - **`AtCompletion`** — cost commits when the built entity attempts to spawn. Allows over-queueing; spawn stalls if cap-blocked. Default for cap-bound resources (pop, supply).
- **Population / cap-bound resources — additive cost direction (Q5).** Each resource also declares `CostDirection` on its catalog entry:
  - **`DeductFromBalance`** — normal stockpile spend. Affordability check: `Balance >= Cost`. Minerals, munitions, fuel.
  - **`AddTowardCap`** — cost adds to the resource's current value, bounded by cap. Affordability check: `Balance + Cost <= Cap`. Pop, supply, any unique faction mechanic that works like pop cap.

  Cost amounts are always positive integers in `FSeinResourceCost`; the resource's `CostDirection` determines the math. Designers author "Marine costs 1 pop" as positive 1; the system interprets it as additive-toward-cap based on the pop resource's declaration.
- **Stall-at-completion semantics.** For `AtCompletion` resources:
  - Build progresses to 100%
  - System attempts spawn via `AttemptSpawn`
  - If any `AtCompletion` resource deduction would exceed its cap, the queue entry **stalls at 100%** — build pending, queue blocked on this item
  - Subsequent ticks retry `AttemptSpawn`
  - When space frees (unit dies, cap raised by a new supply depot, etc.), spawn succeeds and queue advances

  Gives the classic "queued units stall while pop cap is full, spawn when space frees" pattern as an interaction of the existing primitives, not a new primitive.
- **Build time modifiers pre-bake at enqueue (Q4b).** `FSeinProductionQueueEntry::TotalBuildTime` is computed from archetype `BuildTime` + resolved modifier influences at enqueue time, then frozen for that queue entry. Tech granted mid-build does NOT retroactively speed up existing queue items. Matches standard RTS production semantics. Future tech affects future items only.
- **Refund policy — progress-proportional default, opt-in custom percentage (Q2).**

  ```
  USTRUCT(BlueprintType)
  struct FSeinProductionRefundPolicy
  {
      UPROPERTY(EditAnywhere) bool bUseCustomRefund = false;

      UPROPERTY(EditAnywhere, meta=(EditCondition="bUseCustomRefund", ClampMin="0.0", ClampMa
      FFixedPoint CustomRefundPercentage = FFixedPoint::One;
  };
  ```

  - Lives on `USeinArchetypeDefinition`.
  - Default: `refund = (1 - progress_fraction) * cost`. Cancel at 0% = full refund; cancel at 70% = 30% of cost.

  - Opt-in: fixed percentage. `1.0` = full refund; `0.0` = punitive (no refund); anywhere in between for partial-refund patterns.
  - This is a behavior change from the current full-refund default — flagged in implementation deltas.
- **Queue semantics — FIFO, cancel-any-index (Q3a).** Current behavior ratified. No priority reorder, no multi-slot parallel builds. `USeinProductionBPFL` provides `CancelProduction(buildingHandle, queueIndex)`. Refund honored per the archetype's refund policy.
- **Queue-based dependencies — not a framework concern (Q7).** Tech prerequisites validate at enqueue time, not at completion. You cannot queue an item that depends on uncompleted research; this is rejected at enqueue. No mid-queue cascade cancels needed.
- **`ProductionCancelled` visual event** — always fires when a queue entry is cancelled, with the refund amount as payload. UI binds to this for feedback. Similarly, `ProductionStalled` fires when an entry sits at 100% waiting for cap space (for UI to display "Waiting for supply").
- **Rally point extensions (Q9).**
  - Rally target can be a location OR an entity handle. `FSeinProductionData::RallyTarget` becomes a union-ish struct (`FSeinEntityHandle` + `FFixedVector`; `.bIsEntityTarget` flag).
  - On spawn, framework auto-issues a Move command via a CommandBroker (§4) — the produced unit immediately gets a move order to the rally target.
  - No rally chains. A produced unit's move order doesn't inherit or re-trigger rally logic from intermediate points.
- **Abilities can enqueue production (Q10).** `USeinProductionBPFL::SeinQueueProduction(building, archetypeTag, player)` is callable from BP. Ability-driven production (squad reinforce, call-down drops, tech-completion grants) is first-class.
- **Continuous production / auto-requeue — not a framework primitive (Q8).** Designers implement "keep producing X" via an ability on the building that polls queue state and enqueues when empty. Framework stays simple; common patterns script themselves.
- **Production is not special-cased for buildings.** Any entity with `FSeinProductionData` can produce. Ships, factories, airfields, infantry with a "build-a-mine" ability — all use the same system. No "only buildings can produce" invariant.

Non-goals

- Priority-reorder queues. Future extension if common.
- Multi-slot parallel builds (factories producing N things concurrently). Future extension if needed.
- Per-cost-entry deduction timing overrides. Timing is per-resource-definition; if a designer needs "pop-at-enqueue for this one specific building," they define a distinct resource tag with the alternate timing.
- Continuous production as a framework flag. Designers script via abilities.
- Rally chains / rally propagation.
- Mid-build dependency revalidation (tech lost mid-build → queue stays valid; the unit completes).

Implementation deltas that fall out

- Add `FSeinResourceDefinition::CostDirection` (enum) and `FSeinResourceDefinition::ProductionDeductionTiming` (enum) to the catalog.
- `FSeinResourceCost` validation + application logic reads each amount's resource definition to pick the right math (subtract vs add-toward-cap) and timing bucket.
- `USeinProductionSystem` splits cost application into two phases:
  - **At enqueue**: deduct all cost entries whose resources have `AtEnqueue` timing; refund these on cancel per refund policy.
  - **At completion**: attempt to deduct all cost entries whose resources have `AtCompletion` timing; if any

would exceed cap, stall the queue; refund nothing (nothing was deducted yet) if cancelled during stall.

- Add `FSeinProductionRefundPolicy` struct to `USeinArchetypeDefinition`. Update `ProcessCommands::CancelProduction` to honor it.
- Change the default refund behavior from flat 100% to progress-proportional. **Breaking change**: existing content may rely on the old behavior; flag in release notes / migration guide.
- Add `ProductionStalled` visual event + event enum entry. Fire from `USeinProductionSystem` when `AttemptSpawn` fails due to cap.
- Extend `FSeinProductionData::RallyTarget` to support entity-or-location. On spawn, invoke the CommandBroker pathway to issue the move command.
- Wire the production deduction path to share `USeinResourceBPFL::CanAfford` / `Deduct` / `Refund` with ability cost handling (per §5).
- Update `USeinProductionBPFL::SeinGetProductionAvailability` to account for the new two-bucket cost model (can-enqueue-afford vs can-spawn-afford separately).

---

9. Tech / Upgrades `[locked]`

Player progression — tech trees, commander choices, faction upgrades, per-unit refinements. **Not a distinct primitive** — tech *is* an effect (§7) granted by research, scenarios, abilities, or map events, using whichever scope (Instance / Archetype / Player) fits the thing being modified.

The core unification

A tech is a `USeinEffect` subclass. Its **modifier scope** determines where it lives and what it affects:

- **Instance-scope tech** — applied to a single target entity. Lives in that entity's `FSeinActiveEffectsData`. Example: a rifle-upgrade ability that buffs the specific squad that used it; a veteran sergeant attachment adding traits to one unit. Granted via `SeinApplyEffect(entityHandle, TechEffectClass, source)`.
- **Archetype-scope tech** — applied to a player state, influences every owned entity matching an archetype tag at attribute-resolve time. Lives in `FSeinPlayerState::ArchetypeEffects`. Example: "all Infantry +20% damage," "-30% ProductionCost on Vehicles." Granted via `SeinApplyEffect(playerState, TechEffectClass, source)`. Multi-archetype tech is one effect with multiple `FSeinModifier` entries, each targeting a different archetype tag.
- **Player-scope tech** — applied to a player state, modifies player-state-level fields directly. Lives in `FSeinPlayerState::PlayerEffects`. Example: "+10% Manpower income rate," "+50 max population," "-20% Fuel upkeep on all structures." Granted via `SeinApplyEffect(playerState, TechEffectClass, source)`.

A single `USeinEffect` instance carries one scope. Research grants that mix scopes use multiple effects granted together (e.g., research "Mass Production" grants an Archetype-scope effect for "all Infantry builds 20% faster" AND a Player-scope effect for "+10% Manpower income").

Common fields regardless of scope:

- `EffectTag` = the tech tag (e.g., `SeinARTS.Tech.EfficientProduction`). Matches against `PrerequisiteTags` on archetypes and other techs.
- `Modifiers` = stat / field modifiers the tech grants.
- `GrantedTags` = additional tags the tech contributes to the target (entity or player).
- `Duration` = typically `-1` (infinite) for tech; `bRevocable` gates external removal.
- `OnApply` / `OnRemoved` BP hooks for cinematic feedback, research-complete SFX, tech-revoked cleanup.

Research production (§8 with `bIsResearch=true`) becomes: on completion, call `SeinApplyEffect(target, TechEffectClass, source)` where the target depends on scope. Same primitive, same flow. Tech revocation calls `SeinRemoveEffect`.

Upgrades that modify existing units (veterancy, rifle upgrades, armor plating) use Instance scope. Faction-wide bonuses use Archetype. Economy tweaks use Player. No separate "upgrade" primitive.

Decisions

- **Revocability — per-tech flag, default permanent (Q1c).** `USeinEffect` subclasses used as tech carry a `bRevocable` boolean (on the effect config). Default false. Revocable tech enables patterns like mid-match commander-swap or era-regression; permanent tech is the norm for most RTS baselines. The effect system supports removal infrastructure already; the flag gates external remove calls.
- **Source-tracking via explicit BP revoke (Q2b).** The framework does NOT track "which entity granted this tech." If a designer wants building-death-revokes-tech, the building's BP scripts an `OnDeath` handler that calls `SeinRemoveEffect(playerState, techClass)`. Keeps the effect primitive source-agnostic (matches the general principle from §7 that condition sources drive apply/remove themselves).
- **Tech scope is per-tech, not per-match (Q5).** `USeinEffect::TechScope` enum:

  ```
  enum class ESeinTechScope : uint8
  {
      Player,    // Default. Commander choices, faction-specific picks.
      Team,      // Shared radar, alliance-wide intel, co-op tech.
  };
  ```

  On apply, `Player`-scope techs land on the researching player's state; `Team`-scope techs land on every teammate's state (tracked via ally graph).
- **Tiers are emergent, not a primitive (Q4b).** No explicit "Tech Tier 1 / Tier 2" framework concept. `PrerequisiteTags` on archetypes + tech define the structure. Tech-tier chains, phase progressions, era/age advancement all fall out of prerequisite chains.
- **Prerequisites expanded (Q6).**
  - **`PrerequisiteTags`** (existing) — required tags on the player state. All must be present.
  - **`ForbiddenPrerequisiteTags`** (new) — must be *absent* from the player state. Enables "can't research both Tier 2A and Tier 2B" patterns and mutually-exclusive commander choices.
  - **`CanResearch` BlueprintNativeEvent** (new) on `USeinEffect` — escape hatch for BP-scripted eligibility logic (faction-specific, time-gated, map-condition-based). Runs after tag checks; both must pass for the research to be enqueue-able.
- **All granting paths use one API (Q8).** `USeinEffectBPFL::SeinApplyEffect(playerState, TechEffectClass, Source)` is the sole grant entry point. Covers:
  - Research completion (production system calls it).
  - Scenario scripts (§16 scenario event calls it).
  - Abilities (commander call-in ability calls it).
  - Map events (trigger volume's BP calls it).
  - Ally sharing (§17 match settings enable an explicit gift command that calls it).

  No special "grant tech" API — just the effect apply API.
- **Tech revocation affects forward-looking state only (Q10).** Removing a tech from a player:
  - Removes its tag from player-state tag set (refcount ungrant per §2).
  - Removes its modifiers (auto via effect removal — modifiers are the effect's config).
  - Does NOT un-produce units that were conditional on the tech. Existing units stay.
  - DOES cause existing units to lose the archetype-scope stat buffs the tech provided (this is correct — the modifier layer resolves dynamically; remove the modifier, attribute reverts).
  - Fires `TechRevoked` visual event for UI.
- **Tech availability query (Q9).** `USeinTechBPFL::SeinGetTechAvailability(playerState, techClass) →`

`FSeinTechAvailability` returns: `{ bAlreadyResearched, bInProgress, bPrerequisitesMet, bForbiddenPresent, bCustomCheckPassed, bAffordable }`. UI binds uniformly. Matches the shape of ability/production availability queries.

- **Stacking via effect stacking (no new concept).** If a designer wants "research this tech 3 times for +30% damage cumulative," it's just a `MaxStacks > 1` effect. The effect system's stacking rules (§7 Q1) handle it. Standard effect mechanics, no tech-specific plumbing.

Player-state tag unification

With tech-as-effect, the player state needs first-class tag storage the same way entities do (§2). The `FSeinPlayerState::UnlockedTechTags` container becomes a refcounted tag set updated by effect apply/remove hooks. Implementation:

- `FSeinPlayerState` gains `FSeinTagData PlayerTags` (or an equivalent inline refcounted tag container).
- Archetype-scope effects' `EffectTag` + `GrantedTags` grant on apply (refcount++), ungrant on remove (refcount--).
- `HasTechTag(tag)` becomes `PlayerState.PlayerTags.HasTag(tag)` — unified with entity tag queries.
- Existing `HasAllTechTags(container)` stays as a sugar BPFL; delegates to the refcounted set.

Non-goals

- Explicit tier / phase primitive. Emergent from prerequisites.
- Framework-tracked source attribution for tech grants. Designers script revocation hooks.
- Tech-specific cost/time primitives. Uses production (§8) as the granting mechanism with `bIsResearch` + the unified `FSeinResourceCost`.
- Match-settings tech-sharing toggles. Scope is per-tech; if a designer wants "all tech is team-shared in this mode," they re-author the tech effects.
- Retroactive unit removal on tech revoke.
- Dedicated tech tree authoring tool. Designers compose tech trees from prerequisite graphs the same way they compose the rest of their content.

Implementation deltas that fall out

- Add `bRevocable: bool` + `TechScope: ESeinTechScope` + `ForbiddenPrerequisiteTags: FGameplayTagContainer` to `USeinEffect`. Research-specific fields on the effect base class are fine — they're inert for non-tech effects.
- Add `CanResearch` BlueprintNativeEvent to `USeinEffect`.
- Extend `USeinEffectBPFL::SeinApplyEffect` to respect `TechScope` — `Team`-scope effects apply to all teammates' player states.
- Production system (§8): on research completion, call `SeinApplyEffect(playerState, TechEffectClass)` instead of the current `GrantTechTag` + `AddArchetypeModifier` pair. Deprecate those specific paths; keep generic `ApplyEffect` as the canonical way.
- `FSeinPlayerState::UnlockedTechTags` migrates to a refcounted tag container (unified with the entity tag system from §2). `HasTechTag` / `HasAllTechTags` BPFLs delegate through.
- `FSeinPlayerState::ArchetypeModifiers` deprecated — modifiers now live on the archetype effects; `ResolveAttribute` iterates the active effects list instead. Same data, different home.
- Add `USeinTechBPFL::SeinGetTechAvailability` for UI binding.
- Add `TechRevoked` visual event (complement to the existing `TechResearched` event).
- Update CLAUDE.md Pitfalls: "tech is not a separate primitive — it's an archetype-scope effect. Tech changes = effect changes."

---

10. Combat `[locked]`

Combat is NOT a framework pipeline. It is an emergent pattern composed from abilities (§6), effects (§7), attributes, tags, and the deterministic PRNG. Designers implement their game's combat identity (damage types, armor model, accuracy approach, resolution order, cover bonuses, friendly-fire policy, death animations, weapons) in ability scripts and extension components. The framework provides a minimal set of conveniences and invariants that any combat philosophy uses regardless.

This section is short on purpose — it's defining the narrow set of primitives the framework owns, not a combat system.

What the framework provides

- **`FSeinCombatData` — minimal combat component.** Shipped as `USeinCombatComponent`. Carries only:

  ```
  UPROPERTY(EditAnywhere) FFixedPoint MaxHealth = FFixedPoint::FromInt(100);
  UPROPERTY()             FFixedPoint Health    = FFixedPoint::FromInt(100);
  ```

  That's it. Designers extend with additional components (`USeinDynamicComponent` subclasses or their own C++ components) to add Armor, BaseDamage, ShieldPoints, AccuracyRating, or anything else their game needs. Framework only knows about `Health` and `MaxHealth`.

- **Damage / heal BPFLs — convenience layer.** Take final-computed amounts, do the mechanical bits: write the Health delta, fire visual event, update attribution stats, trigger death flow.
  - `USeinCombatBPFL::SeinApplyDamage(Target, Amount, DamageTypeTag, Source)` — writes `FSeinCombatData.Health`.
  - `USeinCombatBPFL::SeinApplyHeal(Target, Amount, HealTypeTag, Source)` — mirror.
  - `USeinCombatBPFL::SeinApplySplashDamage(Center, Radius, Amount, Falloff, DamageTypeTag, Source)` — spatial query + per-entity damage loop. Pure convenience.
  - **None of these compute damage math.** The `Amount` is what the caller passes in after whatever math its game needs. Tooltips explicitly note this.

- **Generic attribute-mutation BPFL — escape hatch.** For custom combat models that don't use `FSeinCombatData`:
  - `USeinCombatBPFL::SeinMutateAttribute(Target, ComponentStruct, FieldName, Delta)` — generic write to any fixed-point attribute on any component. Doesn't fire damage events or update stats. Designer composes their own path.

- **PRNG-roll convenience.** `USeinCombatBPFL::SeinRollAccuracy(BaseAccuracy) → bool`. Uses sim PRNG. Deterministic. Optional — games that prefer deterministic-average damage skip it. Tooltip notes it's a convenience and not framework-integrated elsewhere.

- **Death lifecycle hook.** When `FSeinCombatData.Health` crosses zero (via `SeinApplyDamage` or direct mutation):
  1. Framework checks the entity's ability instances for one tagged `SeinARTS.DeathHandler`.
  2. If present, activates it. The ability runs its OnActivate / latent actions / cleanup and ends.
  3. On ability end (or immediately if no handler), entity is queued for destruction via the existing `DestroyEntity` path.
  4. `Death` visual event fires; `ActorBridge` routes to the actor's `OnDeath` BlueprintImplementableEvent for cosmetic reactions (death animation, particles, SFX).

  Two-hook design: sim-side `DeathHandler` ability for gameplay consequences (split into children, explode, grant XP to killer); render-side `OnDeath` actor event for cosmetics.

- **Attribution stats — tag-keyed, extensible.**

  ```
  USTRUCT(BlueprintType)
  ```

```
struct FSeinPlayerStatsData
{
    UPROPERTY() TMap<FGameplayTag, FFixedPoint> Counters;
};
```

Lives on `FSeinPlayerState`. Framework ships a baseline tag vocabulary in `SeinARTS.Stat.*`:

- `Stat.TotalDamageDealt`
- `Stat.TotalDamageReceived`
- `Stat.TotalHealsDealt`
- `Stat.UnitKillCount`
- `Stat.UnitDeathCount`
- `Stat.UnitsProduced`
- `Stat.UnitsLost`

Framework BPFLs auto-bump these when appropriate (damage BPFL bumps DamageDealt/Received, death flow bumps Kill/Death counts, production completion bumps UnitsProduced).

Designer-extensible. Games add their own stat tags (`MyGame.Stat.MaxHacks`, `MyGame.Stat.WarpJumpsTaken`) and bump them from ability graphs via `USeinStatsBPFL::SeinBumpStat(player, tag, amount)`. Tag-consistent with resources, effects, commands. Stats are sim-side accumulators — deterministic, replay-safe, part of the state hash.

- **Visual events.** `DamageApplied`, `HealApplied`, `Death`, `Kill` — emitted from the framework BPFLs; consumed by UI (floating numbers, hit SFX, death effects). Designers fire additional game-specific events (CriticalHit, ShieldAbsorbed, etc.) from their ability graphs via existing visual event infrastructure.
- **Damage type starter tag.** Ships `SeinARTS.DamageType.Default` only. Designers define their own vocabulary (`MyGame.DamageType.SmallArms`, `MyGame.DamageType.AntiTank`, etc.). No framework damage-type taxonomy.
- **Starter weapon template — aid, not dependency.**

```
USTRUCT(BlueprintType)
struct FSeinWeaponProfile : public FSeinComponent
{
    UPROPERTY(EditAnywhere) FGameplayTag DamageType = FGameplayTag::RequestGameplayTag("Sei
    UPROPERTY(EditAnywhere) FFixedPoint BaseDamage;
    UPROPERTY(EditAnywhere) FFixedPoint Range;
    UPROPERTY(EditAnywhere) FFixedPoint FireInterval;
    UPROPERTY(EditAnywhere) FFixedPoint AccuracyBase;
    UPROPERTY(EditAnywhere) FGameplayTag WeaponTag;
};
```

The framework never reads this struct. It's shipped as a starting point for designers who want a conventional weapon profile shape. Designers can:

- Use it as-is.
- Extend it with additional components (ammo, reload time, barrel count).
- Replace it entirely with their own weapon struct.
- Skip weapons as components and hardcode damage in their attack abilities.

The "multi-weapon unit with independent firing cadences per weapon" pattern (one attack order → several weapons firing at their own rates) is implemented by attaching multiple weapon components to a unit; the attack ability iterates them and spawns per-weapon latent actions. Framework provides the ability + latent-action infrastructure; designers wire the iteration logic.

What designers own

All of the following are explicitly NOT framework concerns:

- **Damage type vocabulary.** Framework ships `DamageType.Default`; game defines the rest.

- **Armor mechanics** (flat / typed / directional / none). Implement via attributes + effects.
- **Accuracy approach** (PRNG / deterministic average / none). Use `SeinRollAccuracy` if you want PRNG, or skip it.
- **Damage resolution order** (which modifiers apply when). Ability script decides.
- **Cover / terrain bonuses.** Tag-queried in attack ability script; effect-applied from terrain zone (§12); whatever pattern the designer wants.
- **Friendly fire policy.** Match setting (§17) exposes a default; ability target filtering enforces. Abilities can override.
- **Death animations / dying-state visuals.** Actor-side, via `OnDeath` BP event + `ActorBridge` visual event routing.
- **Weapons.** Designer-authored components + attack abilities that iterate them.
- **Projectile mechanics.** Designer-authored entities with movement components + `OnHit` abilities + `FSeinLifespanData` for expiration. Framework provides the primitives; designer composes projectiles.

Non-goals

- Hardcoded combat pipeline.
- Framework-provided damage types, armor model, or accuracy system.
- Weapons as a first-class primitive.
- Death state machine beyond the `OnDeath` hook.
- `SeinApplyDamage` computing damage math. It writes a pre-computed delta and fires events.
- Framework awareness of `FSeinWeaponProfile`. Designers iterate their own.
- Event-by-event combat logging in sim state. Attribution accumulators are running totals; event-stream analysis rides on replay re-run.

Implementation deltas that fall out

- Add `MaxHealth` + `Health` fields to the existing `FSeinCombatData` if not already present.
- Add `USeinCombatBPFL` with `SeinApplyDamage`, `SeinApplyHeal`, `SeinApplySplashDamage`, `SeinMutateAttribute`, `SeinRollAccuracy`.
- Add `FSeinPlayerStatsData` component to `FSeinPlayerState` with the tag-keyed counter map.
- Add `USeinStatsBPFL` with `SeinBumpStat`, `SeinGetStat`, `SeinGetAllStats`.
- Register framework stat tags in `SeinARTSGameplayTags.h` (`SeinARTS.Stat.*` namespace).
- Register `SeinARTS.DamageType.Default` in gameplay tags.
- Register `SeinARTS.DeathHandler` ability-classifier tag.
- Add `FSeinWeaponProfile` starter struct (no code references — purely a designer template).
- Wire the damage BPFLs to bump framework stats automatically.
- Wire the death flow: health-zero check in `SeinApplyDamage` → activate death-handler ability if any → destroy via existing `DestroyEntity`.
- `DamageApplied`, `HealApplied`, `Death`, `Kill` visual events added to the visual event enum.
- CLAUDE.md addition under "Pitfalls worth remembering": "Combat is not a framework pipeline. `SeinApplyDamage` writes a pre-computed delta; all math is designer's ability script. Stats are tag-keyed and extensible."

---

11. Vision / LOS / Fog `[locked]`

Per-player or per-team fog-of-war with multi-layer stealth/detection. Deterministic sim-side computation; render-side fog layer reads the latest sim state and presents it to the local player.

Decisions

- **Vision scope is per-player, runtime-mutable.** `FSeinPlayerVisionData` component on `FSeinPlayerState` holds:

  ```
  enum class ESeinVisionScope : uint8 { Private, TeamShared };
  ESeinVisionScope Scope = ESeinVisionScope::Private;
  ```

  Match settings (§17) set the default; tech or abilities can flip at runtime (e.g., intel-themed tech upgrades "Private" → "TeamShared"; a commander-down debuff regresses to "Private").

- **VisionGroups** abstract ownership of the fog state.
  - Each player maps to a VisionGroup ID at any given moment.
  - `Private`-scope player → their own VisionGroup (owned solely by them).
  - `TeamShared`-scope player → shares their team's VisionGroup with other TeamShared teammates.
  - Scope flips at runtime migrate the player's stamp contributions between VisionGroups.
  - Saves memory dramatically in team-games; full FFA still uses N VisionGroups for N players.

- **Vision grid is coarser than nav grid by default.** Plugin setting `VisionCellsPerNavCell` default **4** (vision grid is 1/16 the cell count of nav). Small-map squad-tactical games (~1000^2 nav) can set to 1 or 2 for crisp fog; large-scale massive-unit games (~2000^2+ nav) keep 4 or higher. Re-bake required on change; tooltip explains the trade-off.

- **Cell bitfield: u8 EVNNNNNN per cell per VisionGroup** (on the vision grid, not the nav grid).
  - Bit 0: **E** (Explored, sticky historical — once set, stays set).
  - Bit 1: **V** (Visible, derived from "any layer refcount > 0").
  - Bits 2-7: **N0-N5** (designer-configurable layer visibility bits).
  - Width extensible to u16 (EV + 14 layer bits) via plugin settings if a game needs more than 6 layers.

- **Plugin-settings-declared vision layers.** Designers register layers via:

  ```
  USTRUCT()
  struct FSeinVisionLayerDefinition
  {
      FGameplayTag LayerTag;    // e.g. SeinARTS.VisionLayer.Normal, MyGame.VisionLayer.Therm
      FText DisplayName;
  };
  ```

  `USeinARTSCoreSettings::VisionLayers: TArray<FSeinVisionLayerDefinition>`. Framework ships one default layer (`SeinARTS.VisionLayer.Normal`). Designers add their own (Camouflage, Thermal, DetectorPerception, etc.).

- **Per-layer refcounts per vision-cell per VisionGroup — 4-bit packed nibbles.**
  - Refcount width: **u4 (4 bits) by default, packed 2 refcounts per byte**. Caps at 15 sources per cell per layer — virtually always sufficient for RTS density.
  - Plugin-settings override to u8 (255 cap) if a game genuinely hits the 15 limit.
  - Refcount arrays sized to declared layer count only — 2-layer games don't pay for 6 layers.
  - Bitfield bits V and N0..N5 are **derived** from refcount-layers crossing 0↔1 boundaries. Refcounts are authoritative; bitfield is a cached view for fast reads.

- **Batch-tick update — 4-tick default.** The vision system declares `TickInterval = 4` (runs every 4 sim ticks = 7.5 Hz at 30 Hz sim). Stamp deltas from moved sources are processed in batches, not per-tick. Responsive enough for RTS (below perceptual latency for non-critical UI); saves CPU substantially. Plugin-configurable per-game via `VisionTickInterval`.

- **Tile-partitioned source registry.** Vision sources register to the tile grid (§12). Stamp operations iterate only tiles the source's stamp overlaps. For TeamShared groups, tile contribution is shared. Empty tiles are

skipped entirely — early-game sparse vision hits near-zero cost.

- **Memory at realistic scales** (default 4:1 coarse, 2 declared layers, 4-bit packed refcounts, 8 vision-groups):
  - 1000^2 nav (small-scale squad-tactical 1v1) → 250^2 vision × 8 × (1 bitfield + 1 refcount byte) = **1 MB**
  - 2048^2 nav (standard competitive) → 512^2 vision × 8 × 2 = **4 MB**
  - 5000^2 nav (large-scale with 8 m cells) → 1250^2 vision × 8 × 2 = **24 MB**

  All comfortable.

- **Emission + perception layer model.**
  - `FSeinVisionData` (entity component, used by vision sources):

    ```
    FFixedPoint Radius;
    FGameplayTagContainer EmissionLayers;    // layers this entity emits on; default = {Nor
    FGameplayTagContainer PerceptionLayers;  // layers this entity perceives; default = {Nc
    bool bRequiresLineOfSight = true;
    ```

  - Entity E is visible to VisionGroup G iff: ∃ source S owned by G such that G's cell-bitfield at E's position has at least one bit set from the intersection of S's perception layers with E's emission layers, AND LOS is clear per that layer's blocker rules.

- **Stamp-delta update model.**
  - Each vision source carries a minimal descriptor: `{PreviousCell, Radius, PerceptionLayers, VisionGroup}`.
  - On cell-boundary crossing / radius change / perception-layer change / activation / deactivation: compute new stamp, diff with old, apply refcount deltas to affected cells.
  - No per-source cell-list storage — previous-cell + current-cell + template is enough.
  - Stamps update on actual changes, not per-tick. Most sources (buildings, stationary units) are stamp-once-forever.

- **Template-based vision shape cache.**
  - Precomputed disc templates per unique (Radius, PerceptionLayerSet) combination — stored as cell-offset arrays with LOS baked in assuming no blockers.
  - Stamping an unobstructed source = translate template + apply refcount deltas. O(cells_in_template).
  - When blockers exist in range, fall back to clipped raycast for the affected sector. 80/20: most stamps use templates; obstructed ones raycast.
  - Templates are deterministic (integer grid math).

- **Entity-visibility queries are O(1) via cell lookup.**
  - `SeinIsEntityVisible(viewer, target)` → resolve target's cell → check viewer's VisionGroup's cell bitfield → intersect target's EmissionLayers with the cell's visible layer bits → return bool. No source iteration.

- **Event-driven entity-visibility notifications.**
  - When a stamp delta changes a cell's visibility state, the vision system computes which entities in that cell just entered / exited visibility for that VisionGroup.
  - Fires `EntityEnteredVision(viewerGroup, entity)` / `EntityExitedVision(viewerGroup, entity)` visual events.
  - UI subscribes instead of polling. Minimap pings, threat indicators, targeting reticles all ride on the stream.
  - Pull-query (`SeinIsEntityVisible`) remains as an escape hatch for AI code, ability range checks, and correctness validation.

- **Vision sources.**
  1. **Entity-based** — any entity with `FSeinVisionData` contributes (units, buildings, observation towers).

2. **Ability-spawned** — a scan/recon ability spawns a short-lived vision-only entity (`FSeinVisionData + FSeinLifespanData`). Uses the entity pool (§1); expires on lifespan.
3. **Effect-modified** — effects (§7) can modify a unit's `Radius` or `PerceptionLayers` temporarily.
4. **Terrain-influenced** — high ground and other terrain features grant vision bonuses via terrain query at stamp time (§12).

- **LOS raycasting: discrete grid.**
  - Steps cell-by-cell from source cell to target cell; checks each intersected cell for terrain blockers (from §12 nav grid) and entity blockers.
  - Per-layer LOS: a blocker with `BlockedLayers = {Normal, Thermal}` blocks those layers but not `Radar`.
  - Deterministic integer math.
- **Blockers.**
  - Terrain-level (static map geometry) — baked into the nav grid's terrain data per cell. Always blocks `Normal` by default; designer can extend per-cell block-layer sets via terrain data (§12).
  - Entity-level (dynamic) — `FSeinVisionBlockerData` component with `BlockedLayers: FGameplayTagContainer`. Smoke grenades, destructible forests, constructed walls, ability-spawned cover.
  - Raycasting queries both in a single pass.
- **Stealth / detection via layer interplay.**
  - A camouflaged unit's `EmissionLayers = {Camouflage}` only (not Normal).
  - Regular units' `PerceptionLayers = {Normal}` — can't see the camouflaged unit.
  - Detector units' `PerceptionLayers = {Normal, Camouflage}` — can see it.
  - Temporary detection (a scan ability revealing stealth) is a short-lived vision-source entity with `PerceptionLayers` including Camouflage, covering the scan area.
- **Shared vision propagation.**
  - `TeamShared`-scope players contribute stamps to the team's VisionGroup — all members see what any member sees.
  - `Private`-scope player contributes only to their own VisionGroup.
  - Mixed-scope teams: each Private member has their own group; TeamShared members share the team group. Designer-configurable per-tech and per-match.
- **Render-tick decoupled from sim-tick.**
  - Sim updates the cell bitfield when sources change state (typically sub-tick granularity on movement).
  - Render side (fog of war rendering) ticks at a configurable lower rate — default 10 Hz — reading the latest bitfield snapshot.
  - Entity visibility (for UI/AI) is always current-sim-tick accurate via the event stream + bitfield lookup.
- **Command validity through fog.**
  - Sim resolves the command target at the current sim state, not at client-side "last seen" position.
  - A player right-clicking a ghost image of an enemy at its last-known position issues a command carrying the enemy's entity handle (a real sim-side reference).
  - The ability the command triggers executes at current sim state — the enemy may have moved; attack-move will discover them in their new position.
  - No ghost-filtering or sim rejection on "target-not-visible" — that's a UI-layer hint only.

Future optimization variants (noted, not built)

Locked defaults already include the biggest wins (coarse vision grid, 4-bit packed refcounts, tile partitioning, 4-tick batching). Further variants for extreme scales:

- **Variant C — u16 bitfield for >6 vision layers.** Plugin-settings toggle if a game truly needs more than 6 custom vision layers. Doubles bitfield memory per cell; refcount memory unchanged.
- **Variant D — u8 refcount override.** Designer opt-in to u8 (255-cap) refcounts if 15-cap proves limiting in dense combat scenarios. Doubles refcount memory per cell per layer.
- **Variant E — Per-source LOS cache.** Cache each source's computed visible-cell set across ticks; invalidate only on source move / blocker change. Cuts raycast cost dramatically for stationary vision sources (buildings, towers). Future optimization.

Non-goals

- GPU compute-shader fog. Breaks determinism.
- PVS precomputation. Memory-heavy, poor fit for dynamic blockers.
- CSG / shape-union vision. Scales poorly past ~50 sources.
- Voronoi-cell vision. Expensive to maintain on source movement.
- Per-pixel / continuous raycasting. Discrete grid is deterministic and sufficient.
- Ghost-filtering of commands by viewer fog. Sim is authoritative; fog is a hint.

Implementation deltas that fall out

- `FSeinPlayerVisionData` component on `FSeinPlayerState` with `Scope` field + VisionGroup reference.
- `FSeinVisionGroup` struct (per-group cell bitfield + per-layer refcount arrays).
- `FSeinVisionData` component (entity-side: radius, emission/perception layers, LOS flag).
- `FSeinVisionBlockerData` component (entity-side: blocked layers).
- `FSeinVisionLayerDefinition` struct for plugin settings.
- Framework-shipped layer: `SeinARTS.VisionLayer.Normal`.
- `FSeinVisionSystem` at PostTick — processes stamp deltas, updates bitfields, fires visibility-change events.
- Template cache: `TMap<TPair<FFixedPoint, uint32 LayerHash>, FSeinVisionTemplate>` shared across sources.
- Clipped-raycast path for blocker-obstructed stamps.
- `USeinVisionBPFL`: `SeinIsEntityVisible`, `SeinIsLocationVisible`, `SeinGetVisibleEntities`, `SeinGetCellVisibility`, `SeinGetVisionGroup`.
- `EntityEnteredVision` / `EntityExitedVision` visual events + enum entries.
- Plugin settings: `VisionLayers`, `FogRenderTickRate` (default 10 Hz), `VisionCellsPerNavCell` (default 1, Variant A hook), `VisionTileSize` (default 0, Variant B hook).
- Render-side fog subsystem in `SeinARTSFramework` that ticks at `FogRenderTickRate` and reads the current VisionGroup bitfield for the local player.

---

12. Terrain / Environment `[locked]`

Positional sim state — walkability, elevation, biome, cover, capture points, forests, walls, craters. Layered onto the nav grid + sim entity pool. This section also captures the **nav composition** (HPA* + regional flow fields + formation resolver + steering profiles) and **tile partitioning infrastructure** that serves every grid-backed system.

Two-layer terrain model

- **Layer 1 — Baked static terrain (nav grid cells).** Authored at map bake time from placed volumes + level geometry. Per-cell: walkability, movement cost, optional height/slope, tag indices into a per-map palette. Can be mutated at runtime via BPFL (shelling carves cover tags, designer-scripted events alter walkability).
- **Layer 2 — Dynamic terrain entities.** Buildings, destructible trees, foxholes, constructed walls, craters,

mines, capturable points. Real sim entities (same pool as units) with components: footprint, optional combat/vision/capture, etc. Register their occupied cells with the nav grid on spawn; deregister on destroy. Spawn/destroy flows through the command stream — replay-deterministic.

When is a thing cell-state vs. an entity? Entities have identity, HP, components, lifecycle. Cell-state is pure positional property. Artillery shelling creating cover at impact points might spawn crater entities *and* mutate cell tags — whichever the designer finds clearest.

Compact cell data

Cell data is **the** memory-critical structure at scale. Shape is pluggable via plugin settings:

```
// Default: Flat mode — 4 bytes per cell
USTRUCT()
struct FSeinCellData_Flat
{
    uint8 Flags;              // walkability, cover-type flags, framework bits (8 flag bits)
    uint8 MovementCost;       // 0 = impassable; 1-255 = cost multiplier
    uint8 TagIndices[2];      // palette indices; 0 = null
    // Total: 4 bytes
};

// HeightOnly mode — 6 bytes (padding-aligned)
USTRUCT()
struct FSeinCellData_Height : public FSeinCellData_Flat
{
    uint8 HeightTier;         // 0-255 discrete tier
    uint8 _Pad;
    // Total: 6 bytes
};

// HeightSlope mode — 8 bytes
USTRUCT()
struct FSeinCellData_HeightSlope : public FSeinCellData_Flat
{
    int16 HeightFixed;        // FFixedPoint16 fine-grained
    int8  SlopeX, SlopeY;     // quantized unit-vector slope
    // Total: 8 bytes
};
```

Plugin setting `ElevationMode` picks which struct the grid allocates. Queries go through a virtual interface; games with no elevation don't pay for height fields. Designers may subclass for game-specific extensions.

Default is 4 inline tag indices, not 2. Memory: +2 bytes per cell for 2× fast-path capacity. At 2048² that's +8 MB — worth it. Cells with more than 4 tags spill into a sparse overflow `TMap<int32 CellIndex, TArray<uint8>> CellTagOverflow` on the grid. Rare path.

Per-map tag palette

- `USeinNavigationGrid::CellTagPalette: TArray<FGameplayTag>` — up to 255 unique cell tags per map (u8 index). Baked at map load.
- Cells reference tags by u8 index. `0` is reserved for "no tag."
- Tag queries: `SeinQueryCellTags(cell) → FGameplayTagContainer` resolves indices via the palette.
- If a game needs >255 unique cell tags, plugin-setting toggle switches indices to u16 (adds 4 bytes per cell at 4 inline slots).

Tile partitioning — shared spatial infrastructure

The nav grid is partitioned into **tiles** (default 32×32 cells per tile). Tiles carry summary state used by many systems:

```
USTRUCT()
struct FSeinMapTile
{
    // Entity presence (for range queries / tile skipping)
    TArray<FSeinEntityHandle> OccupyingEntities;

    // Tag union across cells in this tile (dirty-cached)
    FGameplayTagContainer AggregateTags;

    // Vision-source touch bits per VisionGroup (for tile skip in vision pass)
    TBitArray<> VisionGroupActive;   // size = num vision groups

    // HPA* cluster data (see nav composition below)
    int32 ClusterID;
    // ... entrance nodes, cached edge costs
};
```

Every grid-backed system uses the tile grid as broadphase:

- **Vision stamping** iterates only tiles the source's stamp overlaps.
- **Spatial range queries** (`SeinQueryEntitiesInRadius`) iterate tiles in the query's bbox, drill into cells, filter entities.
- **Pathfinding** uses tiles as HPA* clusters.
- **Capture-point polling** uses tile-based entity-in-radius query.
- **Terrain tag aggregate queries** ("any cell in this region has Cover tag?") read the tile's aggregate.

Tile size configurable; 32×32 is a balance between summary-granularity and skipping-benefit. Games with huge maps might use 64×64 tiles.

Nav composition — pathing, formation, steering

Pathing at the scales we target needs three layers working together. Locked as binding requirements for the nav refactor:

*1. HPA for inter-region abstract pathing.**

- Map partitioned into clusters (reuse the tile grid).
- Each cluster has entrance/exit nodes at cluster borders.
- Abstract graph connects exits with precomputed costs.
- Long-distance pathing = A* on the abstract graph (fast, logarithmic in cluster count).
- Returns a sequence of cluster traversals.

2. Regional flow fields for intra-region + group orders.

- One flow field per goal per cluster: each cell in the cluster holds a direction vector toward the goal.
- All units heading to a shared goal read the same field. O(1) per-unit lookup.
- Cheap to compute (Dijkstra sweep over cluster cells; clusters are small).
- Cached per CommandBroker order; recomputed only when goal/members change or cells in the cluster mutate.

3. Formation layer — CommandBroker resolver (§4).

- Given group + goal + formation shape, produces per-member target positions.
- Formation offsets rotate with anchor facing.
- Outputs per-member desired world position, not just "go here."
- Designers author resolvers for open-spacing squad formations, tight-rank formations, convoys, wedge/line/column shapes, etc.

4. Steering — `USeinMovementProfile` subclasses.

- Given current state + target position + flow direction, produces motion respecting physical constraints.

- `USeinInfantryMovementProfile` — tight agility, RVO avoidance, sub-cell pathing.
- `USeinWheeledMovementProfile` — min-turn-radius, no pivot in place, acceleration curves.
- `USeinTrackedMovementProfile` — pivot allowed (slow), heavy mass, acceleration curves.
- Designer-extensible (sea, air, spacecraft, hovercraft profiles). Already scaffolded in the framework.
- Steering consumes flow-field direction as input; formation resolver as position target; physics simulated in fixed-point.

Why all four, not just one:

- Flow field alone = can't abstract long distances, can't represent formations.
- HPA* alone = every unit computes its own path (500 units → 500 queries); no group sharing.
- Formation alone = still need pathing underneath.
- Steering alone = no pathing.

The four compose naturally:

```
Broker order → HPA* picks cluster sequence → regional flow fields cached per cluster
  → Formation resolver computes per-member targets (anchor + offset with facing)
  → Each member's movement profile reads flow direction + target position + current state
  → Motion output in fixed-point
```

This works at both ends of the spectrum:

- **Small-squad tactical games, 1 m cells**: flow fields at 32×32 tile resolution give cell-level direction; Infantry profile with tight RVO handles close quarters.
- **Massive-unit games, 8 m cells, vehicle columns**: flow fields at 32×32 tile = 256 m × 256 m regions; Wheeled profile enforces turn radius; column formation.
- **Large-formation games, 1-2 m cells, 100-member block formation**: flow field for the formation anchor; formation resolver places members in locked ranks; Infantry-TightFormation profile with RVO off.

Cover query — directional vector output

```
USTRUCT()
struct FSeinTerrainCoverQuery
{
    FGameplayTagContainer Tags;            // Cover.Heavy, Cover.Light, custom
    FFixedVector CoverFacingDirection;     // unit vector; {0,0,0} if cover isn't directional
};

// BPFL
FSeinTerrainCoverQuery SeinQueryCoverAtLocation(
    FSeinEntityHandle Defender,
    FFixedVector AttackerLocation
);
```

Designer's attack ability computes:

```
AtkToDefDir = (DefenderLoc - AttackerLoc).Normalize()
DotProduct = FFixedVector::DotProduct(AtkToDefDir, CoverFacingDirection)
// DotProduct in [-1, +1]:
//   +1 = attacking from covered side → full cover bonus
//   -1 = attacking from flanked side → no bonus, possibly flanking penalty
CoverMultiplier = DesignerChoiceScalingFunction(DotProduct, Tags)
```

Framework provides the spatial query + tag data. Designer does the math. No prescription.

Capture points — pluggable capture ability

```
USTRUCT(BlueprintType)
```

```
struct FSeinCapturePointData : public FSeinComponent
{
    UPROPERTY(EditAnywhere, meta=(
        ToolTip="Ability that drives capture logic. Framework ships USeinCaptureAbility_Proximi
                "assign your own ability for action-capture (unit occupies point for a duration
    TSubclassOf<USeinAbility> CaptureAbility;

    UPROPERTY(EditAnywhere) FFixedPoint CaptureRadius;
    UPROPERTY(EditAnywhere) FFixedPoint CaptureTime;

    UPROPERTY() FFixedPoint CurrentProgress;
    UPROPERTY() FSeinPlayerID CurrentOwner;      // Neutral until captured
    UPROPERTY() FSeinPlayerID CapturingPlayer;
    UPROPERTY() TArray<FSeinEntityHandle> Contesters;
};
```

Framework ships `USeinCaptureAbility_Proximity` as the default — passive ability that polls entities-in-radius via `SeinQueryEntitiesInRadius` and advances capture progress. Active-capture (unit stays in place for a duration to capture) is a designer-authored ability. Same pattern as `FSeinWeaponProfile`: starter, not dependency.

Environmental effects — designer-owned

Cells carry tags; designers interpret them. Framework does NOT ship default movement-cost-by-terrain-tag mapping — if a game wants "snow slows movement 30%," it's an effect the designer applies when an entity enters a snow-tagged cell. `Terrain.Environment.Default` is the only framework-shipped cell tag; games extend the vocabulary.

Enter/exit triggers for terrain effects: designer-scripted. Options the framework supports:

- Ability that ticks and reads current cell tags.
- Condition-source entity (cover zone) that polls entities in its radius and applies/removes effects.
- Cell-crossing event stream (if we add one) — future enhancement if useful.

Runtime terrain mutation

BPFLs for mid-match terrain changes:

```
SeinSetCellWalkability(cell, bWalkable)
SeinSetCellMovementCost(cell, cost)
SeinAddCellTag(cell, tag)       // adds to first free TagIndices slot or overflow
SeinRemoveCellTag(cell, tag)
SeinSetCellHeight(cell, height) // if elevation mode supports it
```

Mutations:

- Invalidate affected tiles (trigger flow field / HPA* edge / aggregate-tag recomputation).
- Invalidate vision templates that pass through the cell (LOS may change).
- Fire visual event `TerrainMutated(cells)` for render-side fog/terrain updates.

Example — artillery shelling creating cover at impact: ability at target location calls `SeinAddCellTag(cell, Cover.Light)` on each cell in radius + spawns a crater entity for visual fidelity.

Multi-cell entity footprint

Buildings, walls, large vehicles occupy multiple cells:

```
USTRUCT()
struct FSeinFootprintData : public FSeinComponent
{
    UPROPERTY() TArray<FIntPoint> OccupiedCellOffsets;  // relative to entity position
```

```
    UPROPERTY() FSeinRotation Rotation;                // 0/90/180/270 discrete steps for grid
    UPROPERTY() bool bBlocksPathing = true;
    UPROPERTY() bool bBlocksVision = false;
};
```

On entity spawn: rotate offsets by `Rotation`, compute absolute cell indices, register with nav grid's entity-on-cell map, mark affected cells' pathing/vision flags. On destroy: reverse.

Terrain authoring workflow

Framework provides the data structures; tooling is a later task. For v1, two authoring paths supported:

1. **Placed volumes in Unreal level editor.** Designer-authored blueprint actors that cover areas and declare "cells I cover get tags X, Y" plus "cover facing direction Z" etc. At map bake, volumes are rasterized into cell tags.
2. **Derived from nav.** Existing nav-bake logic extended to populate walkability, height (from geometry raycast), and biome (from material tags).

Deferred: a custom paint-tool Slate widget in SeinARTSEditor for direct cell-tag painting. Significant editor-module investment; queue for post-v1.

Non-goals

- Framework-defined "cover" mechanics. Designer writes cover-math; framework provides spatial queries + tags.
- Framework-defined movement-cost-by-tag. Designer-configured via effects.
- Framework-defined terrain biome semantics. `Environment.Default` only; games extend.
- Continuous (non-grid) LOS raycasting. Discrete grid is deterministic and sufficient.
- Per-cell extension data beyond tag palette. Games extend via the nav grid's sparse overflow or custom cell-struct subclasses.
- Custom paint tool for v1. Defer to editor pass.

Implementation deltas that fall out

- `FSeinCellData_Flat/_Height/_HeightSlope` struct hierarchy via virtual interface.
- `ESeinElevationMode` enum + plugin setting (`None`/`HeightOnly`/`HeightSlope`).
- `USeinNavigationGrid` refactor:
  - Compact cell storage parameterized by `TSubclassOf<USeinCellData>`.
  - Per-map `CellTagPalette: TArray<FGameplayTag>` (u8 or u16 indices).
  - Sparse overflow `CellTagOverflow: TMap<int32, TArray<uint8>>`.
  - Tile grid `Tiles: TArray<FSeinMapTile>` at configurable tile size (default 32).
  - Entity registration API (`RegisterEntity`, `UnregisterEntity`) updating both cells and tile summaries.
  - Runtime mutation BPFLs with dirty-tile propagation.
- `FSeinMapTile` struct with aggregate/summary state.
- `USeinSpatialBPFL::SeinQueryEntitiesInRadius` using tile broadphase.
- HPA* cluster builder over tile grid. Abstract-graph A* pathfinder.
- Regional flow-field computer (Dijkstra sweep per cluster per goal) with per-broker caching.
- `FSeinFootprintData` component for multi-cell entities.
- Rotation handling for footprint placement.
- `FSeinTerrainCoverQuery` struct + `USeinTerrainBPFL::SeinQueryCoverAtLocation` BPFL.
- `FSeinCapturePointData` struct with `CaptureAbility` field.
- `USeinCaptureAbility_Proximity` starter passive ability.
- `SeinARTS.Environment.Default` cell-tag.

- Visual event `TerrainMutated(cells)` for render-side reactions.
- Cell-mutation BPFLs invalidate vision-template cache (§11) for affected cells.
- Volume-based terrain authoring: blueprint actor base class `ASeinTerrainVolume` with `OnBake(navGrid)` hook that rasterizes its declared tags onto covered cells.
- Nav-refactor binding requirements: HPA* + regional flow fields + per-cluster flow-field cache + hierarchical path query API.

Memory examples at realistic scales (Flat cell mode, 4-byte cells, 4-tag inline)

Map scale	Cell size	Grid	Nav memory	Vision (4:1, 2 layers, 8 groups)	Total
Small-scale 1v1	1 m	800^2	2.5 MB	$160^2 \times 8 \times 2$ = 0.4 MB	~3 MB
Standard 2v2	1 m	1500^2	9 MB	$375^2 \times 8 \times 2$ = 2.2 MB	~11 MB
Large competitive	1 m	2048^2	16 MB	$512^2 \times 8 \times 2$ = 4 MB	~20 MB
Large-scale medium	8 m	1500^2	9 MB	$375^2 \times 8 \times 2$ = 2.2 MB	~11 MB
Massive-scale	8 m	5000^2	100 MB	$1250^2 \times 8 \times 2$ = 24 MB	~124 MB

Numbers scale within budget at all RTS sizes; memory is not the limiting factor when cell-size-matches-unit-scale is respected.

---

13. Relationships (containment, transport, garrison, attachment)

`[locked]`

Entity-to-entity edges that aren't CommandBroker membership (§4). Garrison, transport, attachment, crewing. **Hybrid primitive** — a base `FSeinContainmentData` covers common plumbing; specialized components layer on for type-specific semantics.

Core primitive: hybrid base + specializations

```
USTRUCT(BlueprintType)
struct FSeinContainmentData : public FSeinComponent
{
    // Members — the contained entities (may be empty)
    UPROPERTY() TArray<FSeinEntityHandle> Occupants;

    // Capacity
    UPROPERTY(EditAnywhere) int32 TotalCapacity = 1;
    UPROPERTY() int32 CurrentLoad = 0;           // sum of occupant sizes
    UPROPERTY(EditAnywhere) FGameplayTagQuery AcceptedEntityQuery;

    // Behavior on container death
    UPROPERTY(EditAnywhere) bool bEjectOnContainerDeath = true;
    UPROPERTY(EditAnywhere, meta=(EditCondition="bEjectOnContainerDeath"))
    TSubclassOf<USeinEffect> OnEjectEffect;            // optional: applied to each ejected occ
    UPROPERTY(EditAnywhere, meta=(EditCondition="!bEjectOnContainerDeath"))
    TSubclassOf<USeinEffect> OnContainerDeathEffect;   // optional: applied to each occupant if

    // Visibility mode — how occupants interact with the sim while contained
    UPROPERTY(EditAnywhere) ESeinContainmentVisibility Visibility = ESeinContainmentVisibility:

    // Optional deterministic visual-slot assignment (for games where replay visual consistency
    UPROPERTY(EditAnywhere) bool bTracksVisualSlots = false;
    UPROPERTY() TArray<FSeinEntityHandle> VisualSlotAssignments;  // index = visual slot; handl
};
```

```
enum class ESeinContainmentVisibility : uint8
{
    Hidden,              // Occupants excluded from sim spatial queries (garrison, transport tr
    PositionedRelative,  // Occupants tick; position derived from container + slot offset (crew
    Partial,             // Occupants hidden from targeting but contribute to some abilities (e
};
```

Specialization components layer on for type-specific fields:

- `FSeinAttachmentSpec` — named slots with local offsets and accepted-entity queries (for mounted gun crews, hero-joins-regiment, driver/gunner/passenger slots in vehicles).
- `FSeinTransportSpec` — loading/unloading phase timing, deploy points, transit-specific flags.
- `FSeinGarrisonSpec` — firing-slot tags (designer-authored list), window/hatch references for the starter garrison-fire ability.

Specializations are optional. A plain building with infantry garrison can use just `FSeinContainmentData` + a `FSeinGarrisonSpec`. A transport-only vehicle uses `FSeinContainmentData` + `FSeinTransportSpec`. A mounted gun uses `FSeinContainmentData` + `FSeinAttachmentSpec`.

Decisions

- **Orthogonal relationship types per entity.** An entity may have at most **one** attachment AND at most **one** containment simultaneously. They're independent axes.
  - Example: a hero entity is *attached* to a squad; the squad is *contained* in a transport; the transport is *contained* in a freighter. Tree of containments, with an attachment at the leaf.
  - An entity in a CommandBroker (§4) is orthogonal to both — broker membership, attachment, and containment are three independent axes.
- **Containments chain; attachments rarely do but aren't forbidden.** Walking the containment chain from an entity returns its parent, its parent's parent, etc., up to the root (the outermost container). Queries like `SeinGetImmediateContainer(entity)` and `SeinGetRootContainer(entity)` are both provided.
- **Destruction propagation (recursive, configurable).**
  - Container dies with `bEjectOnContainerDeath = true`: immediate occupants are ejected at the container's last position. The ejected entities' own containments (e.g., a squad ejected from a truck might still have a hero attached) are preserved. `OnEjectEffect` applied to each ejected occupant.
  - Container dies with `bEjectOnContainerDeath = false`: occupants die with the container. Recursive — occupants' occupants die too. `OnContainerDeathEffect` applied to each dying occupant before destroy.
  - Designer flexibility: modeling "tank hit hard enough to explode crew" vs "transport driver bails out leaving injured squad" falls out of the flags + effect hooks.
- **Capacity: size + tag query + BP escape.**
  - Container has `TotalCapacity` (integer).
  - Each entity contributes its size to the container's `CurrentLoad` — default 1, overridable via entity's own `FSeinContainmentMemberData::Size` (e.g., a 6-man squad contributes size 6; a tank contributes size 1 to a tank-transporter with capacity 1).
  - Container's `AcceptedEntityQuery` filters which entity tags may enter (e.g., `HasTag(Unit.Infantry)` for infantry-only transports).
  - Designer escape hatch: `CanAccept(Entity) → bool` BlueprintNativeEvent on the container's spec component for any complex case.
- **Visibility modes determine spatial-query behavior.**
  - `Hidden` (garrison, transport in transit): occupants removed from the tile grid's occupant list; excluded from `SeinQueryEntitiesInRadius` and from cell-based vision stamping. They still exist in the entity

pool and their abilities still tick if the designer wants them to.
  - `PositionedRelative` (crew operating a mounted gun, hero attached to a marching regiment): occupants' position is derived from container + attachment-slot offset. They appear in spatial queries at the derived position; they're targetable and visible.
  - `Partial` (rare, e.g., garrisoned infantry that can fire out but can't be directly targeted): designer opts in. Framework treats as Hidden for targeting, Visible for specific ability-triggered fire events.

- **Player-state accounting always includes contained entities.** Pop/supply, unit counts, attribution stats — all count contained entities as owned by their player. A player's 3 infantry squads inside a transport still contribute to their pop; if the transport dies with them, three `UnitDeathCount`s fire (if applicable per `bEjectOnContainerDeath`).

- **Starter abilities for entry/exit.**
  - `USeinAbility_Enter` — generic entry ability. Selected entities targeting a container → ability runs, validates (capacity + query + CanAccept), moves entities to container, hides per visibility mode, registers in occupants list.
  - `USeinAbility_Exit` — generic exit ability. Container's occupants unload at a spawn point (container's deploy point or container position).
  - Designers override these for specific patterns (e.g., airborne paradrop ability = custom exit with descent animation + delayed landing).

- **Player-controller command routing** (designer-authored, but consistent across games):
  - Entities-outside selected + right-click container → `Enter` command if valid.
  - Container selected + right-click outside → `Move` command on the container (its own movement).
  - Occupant selected via UI + right-click outside → `Exit` command on the occupant.

- **Attachment slot system.**

```
USTRUCT()
struct FSeinAttachmentSlotDef
{
    FGameplayTag SlotTag;              // e.g. SeinARTS.Slot.Driver, SeinARTS.Slot.Gunner,
    FFixedVector LocalOffset;           // position relative to container (rotates with fac
    FGameplayTagQuery AcceptedEntityQuery;
};

USTRUCT()
struct FSeinAttachmentSpec : public FSeinComponent
{
    UPROPERTY(EditAnywhere) TArray<FSeinAttachmentSlotDef> Slots;
    UPROPERTY() TMap<FGameplayTag, FSeinEntityHandle> Assignments;  // slot → occupying ent
};
```

  Slot tags drive designer-scripted behavior:
  - Container has a `Slot.Driver` occupant → framework resolver treats container as "driven," enables move abilities.
  - Container has a `Slot.Gunner` occupant → starter `USeinAbility_CrewWeaponOperation` activates the container's weapon via the gunner's commands.
  - Container has a `Slot.Leader` occupant → a tech effect applies morale bonuses.

  Framework stores slots + BPFL for attach/detach. Designer scripts semantics via abilities and effects reacting to slot-occupancy tags on the container.

- **Deterministic visual-slot assignment** (optional sim state).
  - Opt-in via `bTracksVisualSlots` on `FSeinContainmentData`.
  - When an entity enters, framework assigns a visual slot index deterministically (first-free slot, or hash-based stable mapping).

    - `VisualSlotAssignments[i]` = handle of the occupant at visual slot `i`.
    - Render side reads this to position occupant models at specific window/hatch positions in the container actor.
    - For games where replay visual consistency matters (same soldier always fires from the same window across replays), this is essential. For games that don't care, skip the opt-in and save the memory.
- **UI via existing view-model pattern.** Container entity's view-model exposes `GetOccupants`, `GetOccupantCount`, `GetCapacityRemaining`, `GetSlotOccupant(slotTag)`. UI widgets bind normally. No new UI primitive.
- **Starter content under `SeinARTS.Starter.SquadTactics.*` namespace.**
    - `USeinAbility_GarrisonFire` — garrisoned infantry fire weapons from the building's declared firing slots (uses garrison spec + occupant weapon profiles).
    - `USeinAbility_CrewWeaponOperation` — attached crew drives the container's weapon abilities.
    - `USeinAbility_UnitRetreat` — ability that makes the unit fast + bonus survival effect while fleeing to a rally point.
    - Designer uses, extends, or replaces. Framework stays agnostic.

Non-goals

- Framework-prescribed garrison firing mechanics, crew operation protocols, or retreat semantics. Starter abilities exist; designer replaces.
- Chained attachments as a first-class pattern. Framework doesn't forbid them but doesn't optimize for them.
- A unit in two containments simultaneously. One containment + one attachment, orthogonal.
- Rendered occupant visuals as sim concern. Rendering is actor-bridge territory; sim provides the data (occupant list, visual slot assignments) the render side reads.

Implementation deltas that fall out

- `FSeinContainmentData` base struct + `ESeinContainmentVisibility` enum.
- `FSeinContainmentMemberData` on entities that can be contained — carries `Size` (default 1) and a back-reference to current container + slot.
- `FSeinAttachmentSpec`, `FSeinTransportSpec`, `FSeinGarrisonSpec` specialization structs.
- `USeinContainmentBPFL`: `SeinEnterContainer`, `SeinExitContainer`, `SeinAttachToSlot`, `SeinDetachFromSlot`, `SeinGetImmediateContainer`, `SeinGetRootContainer`, `SeinGetOccupants`, `SeinIsContained`.
- `USeinAbility_Enter` / `USeinAbility_Exit` starter abilities (framework-shipped generics).
- `USeinContainmentSystem` at PostTick — processes entry/exit commands, applies visibility mode changes (remove-from-tile-grid / re-add), runs destruction propagation on container death.
- Spatial-query integration: tile grid's occupant-list respects visibility mode (Hidden occupants are excluded from the list).
- Attribution stats bookkeeping: death of a contained entity still counts toward player stats; track via existing `UnitDeathCount` bump in `SeinApplyDamage` death flow.
- Visual slot assignment: deterministic hash or first-free-slot algorithm; stored as sim state when opted in.
- Starter content pack under `SeinARTS.Starter.SquadTactics.*`: `USeinAbility_GarrisonFire`, `USeinAbility_CrewWeaponOperation`, `USeinAbility_UnitRetreat`.
- Visual events: `EntityEnteredContainer`, `EntityExitedContainer`, `AttachmentSlotFilled`, `AttachmentSlotEmptied`.

---

14. Selection & control groups `[locked]`

Client-side ephemeral state for player UI. Not sim. Logged as observer commands (§3) for replay/save UI reconstruction. Control groups are cached selection arrays bound to hotkeys — distinct from CommandBrokers (§4) which are sim-side dispatch entities.

Decisions

- **Selection is client-side only.** No sim-state anywhere on the player state. Lives on `USeinSelectionComponent` on the player controller. Persists across frames on the client; not replicated to other clients.
- **Observer command log is authoritative for replay/save reconstruction.** The framework logs these observer commands in the txn log:
  - `SelectionReplaced { PlayerID, MemberHandles[] }` — full replacement (non-shift click, box-select without modifier).
  - `SelectionAdded { PlayerID, MemberHandles[] }` — additive (shift-click, shift-box-select).
  - `SelectionRemoved { PlayerID, MemberHandles[] }` — subtractive (ctrl-click, ctrl-box-select).
  - `ControlGroupAssigned { PlayerID, GroupIndex, MemberHandles[] }` — bind hotkey to new set.
  - `ControlGroupAddedTo { PlayerID, GroupIndex, MemberHandles[] }` — shift-bind to add to existing group.
  - `ControlGroupSelected { PlayerID, GroupIndex }` — press hotkey to select that group.

  Replay walks this stream; no per-player persistent "selection object" is serialized. Death-driven cleanup (see below) is deterministic on replay without additional observer commands.
- **`bSelectable: bool` on every entity.** Default true. Projectiles, ability-spawned vision stubs, scenario pseudo-entities, etc. set false. Lives on `FSeinEntityData` or as a universal flag (similar treatment to ownership). Stored as a flag, not a tag — tags are for query/inheritance semantics; selectability is a simple yes/no.
- **Selection does NOT require ownership; command issuance DOES.** Two separate permissions:
  - **Selection**: any entity with `bSelectable = true` and visible to the viewing player (not in fog per §11). Can select neutral resource piles, enemy buildings, allied units, etc. — for info-panel display.
  - **Command issuance**: player controller's command-dispatch layer filters the current selection to owned entities (or allied-owned per match settings) before emitting an `ActivateAbility` command. Enemy-selected units are viewable but not orderable.
  - **Broker instantiation enforces wholly-owned invariant** (§4). Even if the controller tries to dispatch a mixed-ownership broker order, the broker primitive filters/rejects. Belt-and-suspenders — the controller filters for UX; the broker enforces as the trust boundary.
- **Control groups are client-side cached selections.** Control group N is a `TArray<FSeinEntityHandle>` bound to a hotkey. Pressing the hotkey issues `ControlGroupSelected` observer command, which the local controller converts to a `SelectionReplaced` with the group's current (post-filtering) members.
- **Transport/containment selection nuance.** When a group member is contained:
  - If the group also contains the member's container → skip the member on move commands (the container carries it). Transport moves, member stays loaded.
  - If the group does NOT contain the container → member disembarks first. Controller emits a `USeinAbility_Exit` pre-command for the member before the move dispatch.

  Framework utility `USeinSelectionBPFL::SeinResolveMoveableSelection(MemberHandles) → (MoveableNow, NeedsDisembark)` partitions the selection. Player controller uses it before dispatching orders.
- **Death-driven group cleanup.** Sim-side entity death fires a `Death` visual event. Each client's `USeinSelectionComponent` listens to the visual event stream; on receipt, iterates its control groups + current selection, removes the dead handle. Deterministic across clients because visual events are deterministic. Replay reconstructs group state by filtering assignments against live entities at playback time — no

additional observer commands needed.

- **Selection through containers exposed via BPFL.**

```
TArray<FSeinEntityHandle> SeinGetOccupants(Container);              // immediate occupants
TArray<FSeinEntityHandle> SeinGetAllNestedOccupants(Container);     // flat, all descendant
FSeinContainmentTree     SeinGetContainmentTree(Container);         // hierarchical tree
```

  UI designer picks the query shape. Top-level-garrison-array pattern reads flat; drill-down pattern reads tree. Framework doesn't prescribe depth or selection UX — just exposes the data.

- **Selection primitive behaviors — framework-shipped.** `USeinSelectionComponent` handles:
  - Replace selection (raw click, box-select without modifier).
  - Add to selection (shift-click, shift-box).
  - Remove from selection (ctrl-click, ctrl-box).
  - Select all by type (double-click an entity → add all entities with matching archetype tag visible on screen).
  - Select all (shortcut, all owned entities on screen).
  - Cycle through subset (next/previous of same type).
  - Bind / recall / add to control group (number-key + shift-number / ctrl-number).

  Designer can override behaviors by subclassing or replacing the component; framework ships default plumbing.

- **AI doesn't select.** AI commands operate on entity handles directly. The selection primitive is purely client/UI. AI's command stream carries member handles explicitly — no intermediate "selection" step.
- **Spectator / replay POV modes.** Replay viewer app decides whether to filter by a specific player's observer stream (authentic player-POV replay with their fog, their selection, their camera) or render everything (spectator mode with free camera, no fog, no selection). Framework provides raw sim state + per-player observer streams; replay app composes modes. Designer-configurable UX; framework stays uninvolved.

Non-goals

- Replication of selection state between clients. Strictly client-local.
- Framework-prescribed selection UX. Starter component + BPFLs cover common cases; designer customizes.
- Cross-player visibility of another player's selection (e.g., in streaming/observer tooling). That's a custom replay-viewer feature, not a framework primitive.
- Server-side selection tracking for cheat prevention. Lockstep + sim-authoritative command validation handles that — selection is purely presentational.
- Selection of non-selectable entities via the framework. `bSelectable=false` is respected; designer can override in BP if they want special cases.

Implementation deltas that fall out

- Add `bSelectable: bool` field to a universal entity component (exists in some form today; check + formalize).
- `USeinSelectionComponent` on the player controller (exists, ratify + extend).
- Observer command types under `SeinARTS.CommandType.Observer.Selection.*` and `...ControlGroup.*`.
- `USeinSelectionBPFL` with: `SeinReplaceSelection`, `SeinAddToSelection`, `SeinRemoveFromSelection`, `SeinSelectAllByType`, `SeinBindControlGroup`, `SeinAddToControlGroup`, `SeinRecallControlGroup`, `SeinResolveMoveableSelection`.
- `USeinContainmentBPFL` getters for nested-occupant queries: `SeinGetOccupants`, `SeinGetAllNestedOccupants`, `SeinGetContainmentTree`.

- Death-visual-event listener on `USeinSelectionComponent` for group-cleanup plumbing.
- Integration with existing `SeinSelectionModel` view-model in the UI toolkit — no changes needed if it already reflects the component state.

Client-side ephemeral state. Who is currently selected, what control groups are bound. Not sim. Logged to the command log as observer commands so that replays can rebuild the UI state of the original player POV.

Decisions aligned:

- `SelectionChanged` and `ControlGroupAssigned`-style commands are observer commands — logged, not processed sim-side.
- Replay can reconstruct "what was the player looking at when X happened" from the command log.

Open questions for Q&A:

- Per-client or per-player (if one player controls multiple units across multiple seats in co-op)?
- Box-select, double-click-all-of-type, type-cycling hotkeys — all same subsystem?
- Persistence across match-session (save/restore selection between games)?

---

15. AI `[locked]`

Strategic AI only. Unit-level micro-AI (dodge, auto-cover, auto-retreat-on-damage) is expressed as passive abilities (§6) with OnTick reactions — not a separate system. This section covers AI opponents and player-equivalent command issuers.

The key insight

Strategic AI sits **above** the sim layer, like a human player. It emits `FSeinCommand`s into the lockstep txn log. In multiplayer, AI runs on exactly one designated client (host by default); its commands are broadcast to all clients via the same lockstep mechanism as human-issued commands. **The AI's internal reasoning does NOT need to be deterministic** — only one machine runs the AI, and only its emitted commands reach the sim. Framework invariants around determinism apply to sim-side code, not to player-level actors (human or AI).

Decisions

- **`USeinAIController` thin base class.** Framework ships a minimal abstract UObject with:
  - `Tick(FSeinTickContext)` — override in BP/C++ to emit commands. Designer picks the tick cadence (every frame, every N sim ticks, wall-clock driven, event-driven).
  - `EmitCommand(FSeinCommand)` — drops a command into the lockstep buffer.
  - Sim-state query helpers (see below).
  - No prescribed architecture — designers build BT, FSM, utility AI, GOAP, scripted, or anything else on top.
- **Two sim-state query flavors.** Both are framework-provided, deterministic, and efficient. Designer picks which fits their AI design:
  - `SeinQueryAllEntities(filter)` — perfect-information query. Ignores fog. Fastest; enables "cheating AI" common in RTS.
  - `SeinQueryVisibleEntitiesForPlayer(PlayerID, filter)` — respects §11 vision state. Used for fog-respecting AI that "plays fair."
  - Both are available; AI author chooses. Framework doesn't enforce a style.
- **AI runs on the designated host client.** In multiplayer: one client is the authoritative AI runner. On host migration (planned/graceful) or dropped-player takeover, the framework spins up the strategic AI on the new host mid-match. Implies:

  - AI controller state is re-createable from current sim state (the new host can pick up where the old one left off; it won't replicate the "thinking" but will make decisions forward from the current state).
  - Short AI absence during migration is acceptable (no commands emitted during the gap).
  - Stateless-resumable AI patterns are preferred; stateful AI needs a snapshot → serialized state passed to the new host at migration time. Design concern for later (match-settings / §17 territory).
- **No framework difficulty primitive.** Difficulty is a designer concern. Multiple AI controller classes, parameter scaling, per-difficulty BP content — all designer choice. Framework provides no Easy/Medium/Hard enum.
- **Faction-specific AI** is designer-authored. Framework makes it easy to author data-driven AI (read from faction data assets, parameterize build orders per faction, etc.) but doesn't prescribe patterns. One-controller-per-faction, data-driven-single-controller, composed AI — all valid.
- **Debug visualization.** AI controllers can log decisions to the regular output log (`LogSeinAI` or `LogTemp`) for dev inspection. Debug drawing of internal AI state (threat maps, planned moves, priority scores) is supported via the regular debug-draw infrastructure. **Debug output MUST NOT enter the command txn log.** Only actual commands cross that boundary. Shipping builds strip debug hooks.
- **Replay.** AI doesn't re-run during replay — the recorded command stream replays deterministically. No special handling. Saving / loading mid-match creates a new timeline forward (as human-player intuition suggests).
- **Starter example content, not framework patterns.** Framework ships an example playable scene with a starter `USeinAIController` subclass authored in Blueprint under `SeinARTS.Starter.*` content — just enough to give designers a working reference. No "blessed" AI architecture. Shipping the framework's opinion on how to build AI is explicitly rejected.

Non-goals

- Framework-prescribed AI architecture (BT, FSM, utility, GOAP).
- Difficulty enum or tuning primitives.
- Per-unit micro AI as a separate system — it's abilities (§6).
- Determinism of AI internal logic. Only emitted commands participate in lockstep.
- Server-authoritative AI simulation (cheating prevention via AI replication). Lockstep + command validation cover that already.
- Sophisticated host-migration AI state transfer. Initial version restarts AI on the new host from current sim state; stateful hand-off is a future enhancement.

Implementation deltas that fall out

- `USeinAIController` abstract UObject base class with:
  - `BlueprintImplementableEvent Tick(FSeinTickContext)`
  - `EmitCommand(FSeinCommand)` BPFL callable from BP
  - `GetOwnedPlayerID()` accessor
- `USeinAIBPFL` with:
  - `SeinQueryAllEntities(FGameplayTagQuery)` for cheating-AI patterns
  - `SeinQueryVisibleEntitiesForPlayer(PlayerID, FGameplayTagQuery)` for fog-respecting AI
  - `SeinEmitCommand(Command)` (forwarded to lockstep)
- `FSeinAIControllerRegistry` on the sim world subsystem — registers/unregisters AI controllers; only the host's instance actively ticks.
- Host-migration hook: on host change, the new host instantiates AI controllers for any AI-owned players. Initial implementation: restart-from-current-state. Future: snapshotted state hand-off.
- Dropped-player takeover: same plumbing — if a human player disconnects and match settings enable AI takeover, an AI controller is instantiated on the host to continue their play.

- Debug logging: `DEFINE_LOG_CATEGORY_EXTERN(LogSeinAI, Log, All)`. AI authors log through it. Debug draw via existing editor debug-draw.
- Starter example AI: a `USeinAIController_Starter` BP class under `SeinARTS.Starter.*` content, demonstrating simple production + attack-move patterns. Ships with the example playable scene.

Server/host-resident command issuer. Issues `FSeinCommand`s the same way a human player controller would. Replays and multiplayer-in-lobby work automatically because AI commands flow through the same lockstep buffer.

Decisions aligned:

- AI is a player-equivalent command issuer, not a special sim primitive.
- Runs on the host in networked games; runs locally in single-player.

Open questions for Q&A:

- Decision-making scope: behavior tree, state machine, utility AI, scripted?
- How does AI query sim state? Same read APIs as the UI layer, or a dedicated AI "blackboard" abstraction?
- Difficulty scaling — via modifiers on the AI's issued commands (e.g., lower APM), via per-difficulty resource bonuses, or both?
- Faction-specific AI personalities?

---

16. Scenario events / cinematics `[locked]`

Co-op campaigns, mission scripting, cinematics, objectives, scripted events. **Not a new primitive** — scenarios compose from entities + abilities + tags + effects + visual events, with a thin utility layer for the patterns that recur (cinematic playback, sim pause, named entity lookup).

The core pattern

A scenario is an abstract sim entity (per §1, no transform meaningful) with an `FSeinAbilitiesComponent` holding **scenario abilities**. Scenario abilities are ordinary `USeinAbility` subclasses — a passive "OnScenarioTick" poller + response abilities that other abilities call into as messages.

- Passive `OnScenarioTick` reads sim state each tick (or batched per `TickInterval`), checks conditions, branches into `SeinActivateAbility(self, Tag.Response.X)` calls to trigger scripted responses.
- Response abilities emit commands, apply effects, fire cinematic visual events, grant tech, update tags, etc.
- Other abilities (capture completions, unit deaths, mission-critical events) call the scenario's response abilities directly via `SeinActivateAbility(scenarioHandle, Tag.Response.X, payload)`.

This uses only existing primitives. No `USeinScenario` UObject. No event-subscription system. Just entity + abilities + tag queries + registry lookups.

Decisions

- **Scenarios are abstract sim entities.** Spawned like any entity via `SpawnEntity`; no render-side actor needed (set `bIsAbstract=true` on the `USeinArchetypeDefinition`). The scenario entity has tags, ability components, effects — full sim-state participation.
- **Abstract-entity support via `bIsAbstract` flag.** Added to `USeinArchetypeDefinition`. When true: `USeinActorBridgeSubsystem` spawns the backing `ASeinActor` with `bHidden=true` and skips cosmetic visual-event routing. Archetype metadata (tag, display name) still populated for debug/inspection. Overhead negligible.
- **Entity lookup via framework-level infrastructure.** Scenarios use `USeinEntityLookupBPFL` (framework primitive, per §2 + invariant #10) for named singleton lookup and tag-based set lookup. No scenario-specific registry. Typical scenario patterns:

  - Singleton: on scenario spawn, call `SeinRegisterNamedEntity("MissionScenario", self)`. Any ability anywhere calls `SeinLookupNamedEntity("MissionScenario")` to get the handle.
  - Set query: `SeinLookupEntitiesByTag(Unit.Rifleman)` returns all Rifleman entities in the match, no iteration. Auto-indexed by the tag refcount system.
  - Mission-critical unique entities: tag them with a unique tag (`Mission.Hero.Bob`) and use `SeinLookupFirstEntityByTag` — or register by name if the tag isn't granted elsewhere.

- **Message pattern for entity-to-scenario signaling.** Capture completes → capture ability looks up the scenario handle via registry → `SeinActivateAbility(scenarioHandle, Tag.Response.CapturePointTaken, capturePointHandle)`. Scenario's response ability activates with the payload in its `TargetEntity` / `TargetLocation` / `FInstancedStruct Payload` fields (per §3 command shape). No new messaging primitive — it's ability activation with a payload.

- **Cinematic support — three BPFLs + two visual events + a skip-mode enum.**
  - `USeinScenarioBPFL::SeinSetSimPaused(bool bPaused)` — global sim pause. Pauses the tick loop; commands still accumulate in the txn buffer but don't process until resumed. Used by blocking cinematics; designer choice per-trigger in their scenario graph.
  - `USeinScenarioBPFL::SeinPlayPreRenderedCinematic(TSoftObjectPtr<UMediaSource> Asset, bool bBlocksSim, ESeinCinematicSkipMode SkipMode, int32 VoteThreshold)` — fires a `PlayPreRenderedCinematic` visual event with the parameters. If `bBlocksSim`, pauses the sim first.
  - `USeinScenarioBPFL::SeinEndCinematic(FName CinematicID)` — fires `EndCinematic` visual event, resumes sim if paused. Called by scenario when cinematic ends or skip threshold reached.
  - Visual events: `PlayPreRenderedCinematic(assetRef, skipMode, voteThreshold)`, `EndCinematic(cinematicID)`.
  - Skip modes:

    ```
    enum class ESeinCinematicSkipMode : uint8
    {
        Individual,   // Each player's local video stops on their skip. Non-blocking-sim on
        VoteToSkip,   // Uses §17 voting primitive; threshold reached → EndCinematic fires
        HostOnly,     // Only host can skip.
        NoSkip,       // Skip input ignored.
    };
    ```

  - For `VoteToSkip`: forward-references §17 voting primitive. Cinematic-start spawns a `Vote.SkipCinematic.{cinematicID}` vote context. Players submit `CastVote` commands via the skip button. When threshold met, `OnVoteResolved` fires and scenario's listener calls `EndCinematic`.

- **Deterministic cinematic coordination.** Video playback is render-side (wall-clock). Sim-side is deterministic via the pause + sim-tick of the EndCinematic event. All clients see sim resume at identical tick when a blocking cinematic ends. Per-client wall-clock variance in video playback duration is tolerable — clients with faster video catch up to the sim-tick resume together.

- **Multiple scenarios composable.** A match can have N scenario entities (main mission + side objective + tutorial overlay + whatever). Each has its own ability graph + state. Events/messages route to whichever scenario registered for them; no framework orchestration needed between them.

- **Save/resume via command stream replay.** No explicit scenario state serialization. Save = command log up to tick N. Resume = replay commands 0→N and continue. Scenario state re-derives from the deterministic sim.

- **Objective primitive NOT shipped.** Objectives vary too widely across games (kill-count, capture-all, survive-N-minutes, escort, defend-location, branching-optional). Designers compose objectives from scenario abilities + tags + UI view-models. Framework ships no objective struct or system.

- **Starter example scenario under `SeinARTS.Starter.*` namespace.** One minimal scenario BP (maybe a "kill all enemies" pattern) to give designers a reference. Not a blessed pattern.

What gets used from other sections

- §1 Entities — scenario is an abstract entity.
- §6 Abilities — scenario logic IS abilities.
- §7 Effects — scenarios grant effects (unlock tech, apply buffs to mission units, trigger story-driven debuffs).
- §3 Commands — scenario abilities emit commands; scenarios can themselves be command issuers (like strategic AI).
- §11 Vision — scenarios reveal/hide map regions via vision-source entities spawned/despawned.
- §12 Terrain — scenarios mutate terrain, spawn/destroy obstacle entities.
- §17 Match Settings — voting primitive for skip coordination; cinematic defaults; match-flow events.

Non-goals

- `USeinScenario` UObject as a new primitive.
- `TriggerEventTags` on abilities (retracted — tick-polling + external activation sufficient).
- Framework-prescribed objective primitive.
- Scenario state serialization beyond command-stream replay.
- In-editor scenario timeline editor / visual trigger designer. BP graphs are the authoring surface.
- Cross-scenario orchestration primitives (scenarios are independent; if they need to coordinate, designer wires messages between them via registry lookups).

Implementation deltas that fall out

- Add `bIsAbstract: bool` field to `USeinArchetypeDefinition` (default false).
- `USeinActorBridgeSubsystem`: on entity spawn with `bIsAbstract=true`, spawn actor hidden + skip cosmetic visual event routing.
- Scenario code uses the framework-level `USeinEntityLookupBPFL` (defined in the §2 implementation deltas) for named + tag-based lookups. No scenario-specific lookup primitive.
- `USeinScenarioBPFL` with `SeinSetSimPaused` + cinematic helpers.
- `USeinWorldSubsystem::bSimPaused` flag respected by tick loop (paused = commands accumulate but don't process; visual events still flush to render).
- `PlayPreRenderedCinematic` / `EndCinematic` visual events added to the visual event enum.
- `ESeinCinematicSkipMode` enum.
- Integration with §17 voting for `VoteToSkip` mode (forward-reference).
- Rename `SeinActorFactory` → `SeinEntityFactory` (minimal, kept as-is functionality).
- Add `SeinUnitFactory` (pre-seeds Archetype + Bridge + Abilities + Movement + Combat components).
- Starter scenario BP under `SeinARTS.Starter.*` content.
- Example playable scene wiring: starter scenario + starter AI (§15) + example units.

Co-op campaign scripting. "When X dies, spawn Y and reveal Z." "Cinematic at tick N triggers dialogue and camera pan." Must play in sync across all co-op clients.

Decisions aligned:

- Scripted events must play deterministically in sync across co-op clients. Implies: either they ride on the sim (sim state changes trigger them), or they are themselves commands in the lockstep stream.
- Cinematics in particular need deterministic trigger points + synchronized playback.

Open questions for Q&A:

- Authoring model: Blueprint graph, dedicated asset (`USeinScenario`), or in-level actors with event volumes?
- Runtime: a dedicated subsystem that ticks with the sim, reading sim state and emitting commands/visual events? Or a set of abilities granted to a scenario-owner entity?
- Interaction with replay: scenario scripts should replay identically because their inputs (sim state) replay

identically.

- Mid-mission save/resume (if that's on the roadmap)?

17. Match Settings & match flow `[locked]`

Match-level configuration and flow infrastructure. Everything that's "not a sim primitive but affects how a match plays out": resource sharing, friendly fire, voting mechanics, pause modes, host migration, victory triggers, replay headers, diplomacy, match state machine.

Philosophy — framework enforces at choke points, composes above

This section captures a lot of mechanisms, but the framework's role is narrow: provide the **infrastructure** for match flow + voting + replay + match-level configuration, and let designers compose **policy** (victory conditions, game modes, objective systems, UI) on top.

Framework-enforced choke points:

- Match settings are immutable after match start (snapshot into the sim; desync-safe).
- The match state machine transitions are command-driven (lockstep-deterministic).
- Voting is a native sim primitive (enforced tallying + resolution).
- Replay headers capture enough metadata to re-run a match deterministically.
- Host migration triggers are framework-managed (new host picks up; AI takeover where configured).

Composed-above-choke-points (designer freestyle):

- Victory conditions (annihilation, objectives, timer, co-op survival — scenarios decide).
- Game modes (skirmish, campaign, co-op, custom — designer-orchestrated).
- Lobby UI + match settings UI (designer-authored per game's UX).
- Diplomacy mechanics beyond the relation primitive (treaties, trade, pacts — scenario logic).

Decisions

- **Storage: `FSeinMatchSettings` USTRUCT + `USeinMatchSettings` UObject assets + `FInstancedStruct CustomSettings` for extensibility.**
  - `FSeinMatchSettings` is the sim-side runtime struct with framework-shipped fields.
  - `USeinMatchSettings` is a designer-authored data asset holding a preset (`FSeinMatchSettings` default values + a name/icon for lobby UI).
  - Designers add their own settings via a custom USTRUCT embedded in `FInstancedStruct CustomSettings`. Framework doesn't read it; designer BPFLs do.

```
USTRUCT(BlueprintType)
struct FSeinMatchSettings
{
    // Resource & combat
    ESeinResourceSharingMode ResourceSharing = ESeinResourceSharingMode::PerPlayer;
    bool bAlliedCommandSharing = false;
    bool bFriendlyFire = false;

    // Match flow
    FFixedPoint PreMatchCountdown = FFixedPoint::FromInt(3);
    FFixedPoint MinMatchDuration = FFixedPoint::Zero;  // 0 = no minimum
    ESeinPauseMode DefaultPauseMode = ESeinPauseMode::Tactical;

    // Host / drop handling
    ESeinHostDropAction HostDropAction = ESeinHostDropAction::PauseUntilNewHost;
    bool bEnableAITakeoverForDroppedPlayers = false;
```

```
    // Diplomacy — tag-based, see full section below
    bool bAllowMidMatchDiplomacy = false;  // if false, diplomacy frozen at match start

    // Spectator
    bool bAllowSpectators = true;

    // Designer extensions — opaque to framework
    UPROPERTY() FInstancedStruct CustomSettings;
};
```

- **Immutable after match start.** Settings are snapshot from lobby into `USeinWorldSubsystem::CurrentMatchSettings` at `StartSimulation()` time. Mid-match mutation is forbidden (would desync replays). Access via `USeinMatchSettingsBPFL::Get() → FSeinMatchSettings`.
- **Match flow state machine.**

```
enum class ESeinMatchState : uint8
{
    Lobby,      // Pre-match, players joining, settings configurable
    Starting,   // Countdown ticking, commands blocked
    Playing,    // Normal sim execution
    Paused,     // Sim halted (per pause mode)
    Ending,     // Victory/defeat declared, cleanup phase
    Ended,      // Match over, replay saved, ready to return to lobby
};
```

State transitions are command-driven: `StartMatch`, `PauseMatchRequest` (votes), `ResumeMatchRequest`, `EndMatch(winner, reason)`, `ConcedeMatch`, `RestartMatch`. Each transition fires a visual event for UI/scenario subscription.

- **Pause modes.**

```
enum class ESeinPauseMode : uint8
{
    Tactical,   // Sim halts; commands accumulate; resume replays them. Default for campaig
    Hard,       // Sim halts; commands rejected during pause. Cleaner for competitive-vote
};

// BPFL
SeinSetSimPaused(bool bPaused, ESeinPauseMode Mode = Tactical);
```

`SeinSetSimPaused` from §16 gains the mode parameter. Default comes from `FSeinMatchSettings::DefaultPauseMode`; per-call override possible (e.g., campaign cinematic forces Tactical; PvP vote-to-pause uses Hard).

- **Voting primitive.** First-class sim state — supports cinematic skip, concede, pause requests, restart, kick-player, campaign decisions.

```
USTRUCT()
struct FSeinVoteState
{
    FGameplayTag VoteType;                          // "Vote.SkipCinematic.ID_123", "Vote.C
    TMap<FSeinPlayerID, int32> Votes;               // 0=no, 1=yes; custom semantics via de
    int32 RequiredThreshold;                         // N yes-votes needed (or use Resoluti
    ESeinVoteResolution Resolution;                  // Majority / Unanimous / HostDecides
    int32 InitiatedAtTick;
    int32 ExpiresAtTick;                             // auto-resolve (fail) if timer expire
    FSeinPlayerID Initiator;
};

// Commands
FSeinCommand::StartVote(voteType, resolution, threshold, expiresInTicks)
FSeinCommand::CastVote(voteType, voteValue)
```

```
// Visual events
OnVoteStarted(voteType, initiator, threshold)
OnVoteProgress(voteType, yesCount, noCount, totalEligible)
OnVoteResolved(voteType, bPassed, finalVotes)

// BPFLs
USeinVoteBPFL::SeinStartVote(voteType, resolution, threshold, expiresInTicks);
USeinVoteBPFL::SeinCastVote(voteType, voteValue);
USeinVoteBPFL::SeinGetActiveVotes() → TArray<FSeinVoteState>;
USeinVoteBPFL::SeinCheckVoteStatus(voteType) → (Active/Passed/Failed/NotStarted);
```

Designers use votes for: skip cinematics (§16), concede, pause-request, kick-player, restart-mission, campaign decision-gates. Fully tag-keyed and extensible.

- **Victory is scenario-driven, not a framework primitive.** Framework provides:

```
USeinMatchFlowBPFL::SeinEndMatch(FSeinPlayerID Winner, FGameplayTag Reason);
```

  Scenarios or match-flow code call this when their game-specific victory condition fires. Framework emits `MatchEnded(winner, reason)` visual event; transitions state machine to `Ending` → `Ended`. No framework-prescribed victory enum.

- **Spectator support.**
  - Players register with `bIsSpectator: bool` on `FSeinPlayerState`.
  - Spectator command filter in `ProcessCommands`: spectators can only emit observer commands (camera, chat, ping). Sim-mutating commands from spectators are rejected + logged.
  - Replay viewers are essentially spectators of a recorded command stream.

- **Host migration.**

```
enum class ESeinHostDropAction : uint8
{
    EndMatch,               // Match ends immediately on host drop.
    PauseUntilNewHost,      // Sim pauses until new host elected.
    AutoMigrate,            // New host auto-elected (typically lowest-latency-remaining).
    AITakeover,             // Auto-migrate + spawn AI for any dropped player's entities.
};
```

  Match setting selects behavior. Framework provides the migration plumbing; host election logic + network-layer handshake are UE networking concerns the framework orchestrates.

- **Diplomacy — tag-based, not an enum.**

  A 3-state enum (Enemy/Neutral/Allied) was too restrictive for the richer diplomacy models common in grand-strategy / 4X / some RTS titles (cold war, non-aggression pacts, defensive alliances, trade agreements, vassal/tributary, etc.). Diplomacy is modeled as a **directional `FGameplayTagContainer` between each pair of players**.

```
// Per-pair directional storage on USeinWorldSubsystem
TMap<TPair<FSeinPlayerID, FSeinPlayerID>, FGameplayTagContainer> DiplomacyRelations;
// Key (A, B) = "from A's perspective toward B". Directional because many relations are asy
// (A unilaterally declares war; B consents to peace; A grants military access independentl
```

  Framework-shipped baseline vocabulary in `SeinARTS.Diplomacy.*`. Framework systems reference specific tags for cross-cutting behavior (friendly-fire, targetability, shared-vision). Designer games extend freely.

```
SeinARTS.Diplomacy.State.AtWar          // mutually hostile; default-targetable
SeinARTS.Diplomacy.State.Peace          // neutral, no conflict
SeinARTS.Diplomacy.State.Truce          // temporary peace, auto-expires
SeinARTS.Diplomacy.Permission.Allied           // treated as ally for combat / command
SeinARTS.Diplomacy.Permission.SharedVision     // vision propagates (§11 TeamShared equival
```

```
SeinARTS.Diplomacy.Permission.OpenBorders      // units may traverse territory
SeinARTS.Diplomacy.Permission.ResourceShare    // resource pool shares (see §5 sharing mode
SeinARTS.Diplomacy.Permission.CommandSharing   // gates §4 allied-command invariant
```

Designers extend (`MyGame.Diplomacy.Treaty.NonAggressionPact`, `MyGame.Diplomacy.Status.Vassal`, etc.) without touching framework code.

Framework-gated semantics (the choke points):

- `Permission.Allied` → §4 broker ownership loosening (if `bAlliedCommandSharing` match-setting is on).
- `Permission.SharedVision` → §11 VisionGroup aggregation.
- `Permission.Allied` + `bFriendlyFire=false` → §10 damage filter skips allied targets.
- `State.AtWar` → targetability default-hostile.

Everything else is designer territory: treaty negotiation, AI diplomacy preferences, war-declaration consequences, reputation systems, treaty breach penalties.

Mutation is command-driven (deterministic):

```
FSeinCommand::ModifyDiplomacy {
    FSeinPlayerID FromPlayer;
    FSeinPlayerID ToPlayer;
    FGameplayTagContainer TagsToAdd;
    FGameplayTagContainer TagsToRemove;
}
```

`USeinDiplomacyBPFL` surface:

```
// Query
FGameplayTagContainer  SeinGetDiplomacyTags(From, To);  // directional lookup
bool                   SeinHasDiplomacyTag(From, To, Tag, bBidirectional=false);
TArray<FSeinPlayerID>  SeinGetPlayersWithDiplomacyTag(From, Tag);

// Mutation (routes through the command buffer — not direct write)
void  SeinModifyDiplomacy(From, To, TagsToAdd, TagsToRemove);

// Convenience wrappers for baseline state transitions
void  SeinDeclareWar(From, To);        // adds AtWar, removes Peace/Truce (framework conven
void  SeinDeclarePeace(From, To);      // adds Peace, removes AtWar
void  SeinProposeTruce(From, To, Duration);  // adds Truce with expiration

// Cross-cutting framework-system helpers (bidirectional checks)
bool  SeinAreAllied(playerA, playerB);
bool  SeinAreAtWar(playerA, playerB);
bool  SeinHasSharedVision(viewer, target);  // consults diplomacy + §11 VisionScope
```

Visual event fires on every `ModifyDiplomacy` application:

```
OnDiplomacyChanged(FromPlayer, ToPlayer, TagsAdded, TagsRemoved, FullTagsAfter, Tick)
```

Scenarios, UI, and AI subscribe. Replays fire the same events at identical ticks because the command stream is deterministic.

Parallel tag index (mirrors §2's entity-tag-index pattern): `USeinWorldSubsystem::DiplomacyTagIndex: TMap<FGameplayTag, TArray<TPair<FSeinPlayerID, FSeinPlayerID>>>`. Auto-maintained by the `ModifyDiplomacy` command handler. Enables O(1) queries like "find all pairs with `State.AtWar`." Memory trivial (≤ N_players² × a few tags).

`FSeinMatchSettings::bAllowMidMatchDiplomacy` gates runtime mutation. If false, the diplomacy command handler rejects mid-match `ModifyDiplomacy` commands (diplomacy locked at match start — typical for team-based competitive). If true, mid-match declare-war / treaty / alliance-swap are possible (grand-strategy / campaign).

- **Pre-match countdown.**
  - Match state `Starting` runs for `PreMatchCountdown` seconds (configurable).
  - Sim doesn't accept player commands during this phase (players see "Starting in 3, 2, 1...").
  - On countdown complete, state transitions to `Playing`; scenarios fire `OnMatchStarted`.
- **Match flow events** (fired by framework, consumed by scenarios/UI):
  - `OnMatchStarting` — entered Starting state, countdown begins
  - `OnMatchStarted` — transitioned to Playing
  - `OnMatchPaused(reason)` — sim paused
  - `OnMatchResumed` — sim resumed
  - `OnMatchEnding(winner, reason)` — EndMatch called
  - `OnMatchEnded` — cleanup complete, ready for next match
  - Plus the player-level events from §16 Q4 expansion (registered/dropped/ready/surrendered/voted).
- **Replay header + serialization.**

  ```
  USTRUCT()
  struct FSeinReplayHeader
  {
      FString FrameworkVersion;
      FString GameVersion;
      FName MapIdentifier;
      uint64 RandomSeed;
      FSeinMatchSettings SettingsSnapshot;
      TArray<FSeinPlayerRegistration> Players;  // PlayerID, Faction, Team, IsAI/IsSpectator
      int32 StartTick;
      int32 EndTick;
      FDateTime RecordedAt;
  };

  // File format
  // [Header (JSON or Unreal-serialized)][CommandLog (binary, tick-grouped)]
  ```

  Framework ships `USeinReplayBPFL::SeinSaveReplay(path)` / `SeinLoadReplay(path)`. Replay playback = load header → register players → feed command log at recorded ticks → sim re-runs deterministically.
- **Game modes are designer compositions, not a framework primitive.** Each game's `AGameModeBase` subclass selects a `USeinMatchSettings` preset, spawns the appropriate scenario (§16), configures faction assignments, wires lobby UI. Framework provides the settings + scenario + voting infrastructure; designer assembles.

Non-goals

- Framework-prescribed victory conditions, objective systems, or game mode enums.
- Mid-match mutation of core match settings.
- Framework-shipped lobby UI.
- Network-layer host election logic (UE networking handles; framework orchestrates around it).
- Designer-locked vote types — games can invent new `Vote.*` tags freely.
- Sim-speed multipliers in v1 (pause/play only; speed-up for single-player/skirmish is flagged as future enhancement).

Implementation deltas that fall out

- Add `FSeinMatchSettings` USTRUCT with all fields listed above.
- Add `USeinMatchSettings` UObject asset class for lobby-presetting.
- `USeinWorldSubsystem::CurrentMatchSettings` + snapshot-at-StartSimulation flow.

- `USeinMatchSettingsBPFL` for runtime read access (no write after match start).
- `ESeinMatchState` enum + state machine on `USeinWorldSubsystem`.
- Match flow commands: `StartMatch`, `PauseMatchRequest`, `ResumeMatchRequest`, `EndMatch`, `ConcedeMatch`, `RestartMatch`.
- `FSeinVoteState` storage on `USeinWorldSubsystem` + `USeinVoteBPFL` with start/cast/check/get BPFLs. Auto-expire votes via a sim system ticking votes.
- Vote visual events: `OnVoteStarted`, `OnVoteProgress`, `OnVoteResolved`.
- `ESeinPauseMode` enum; update `SeinSetSimPaused` signature (§16 revision).
- `ESeinHostDropAction` enum + migration plumbing hooks.
- Tag-based diplomacy storage: `TMap<TPair<FSeinPlayerID, FSeinPlayerID>, FGameplayTagContainer> DiplomacyRelations` on `USeinWorldSubsystem`.
- Parallel `DiplomacyTagIndex: TMap<FGameplayTag, TArray<TPair<FSeinPlayerID, FSeinPlayerID>>>` auto-maintained by the `ModifyDiplomacy` command handler.
- `USeinDiplomacyBPFL` with directional query BPFLs, bidirectional check helpers, convenience wrappers (DeclareWar, DeclarePeace, ProposeTruce), and `ModifyDiplomacy` mutation.
- Register framework diplomacy tags: `SeinARTS.Diplomacy.State.{AtWar, Peace, Truce}` + `SeinARTS.Diplomacy.Permission.{Allied, SharedVision, OpenBorders, ResourceShare, CommandSharing}`.
- `FSeinCommand::ModifyDiplomacy` command type + `ProcessCommands` handler that respects `bAllowMidMatchDiplomacy`.
- `OnDiplomacyChanged` visual event.
- Integration points: §4 broker owner-scope checks `Permission.Allied`; §10 damage filter checks `Permission.Allied` vs `bFriendlyFire`; §11 VisionGroup aggregates across `Permission.SharedVision` relations.
- `USeinMatchFlowBPFL::SeinEndMatch` + match flow visual events.
- Spectator command filter in `ProcessCommands`.
- `FSeinReplayHeader` struct + `USeinReplayBPFL::SeinSaveReplay` / `SeinLoadReplay`.
- `AGameModeBase`-compatible integration point so designer game modes pick up the framework's match flow.

Open cross-system questions (to resolve during Q&A)

- **Default component set.** Tabled for now per discussion — revisit if composition cost becomes a pain point.
- **Reference counting for shared tag state.** Resolved in §2 via refcount model.
- **Whether `USeinArchetypeDefinition` itself should shrink** as things like `DefaultCommands` move off it. Likely yes — it ends up carrying identity, cost, tech metadata, archetype-scope modifier declarations, and (per §1) `BaseTags`.
- **Match settings primitive.** Resolved in §17.
- **Sim-speed multipliers.** Flagged as future enhancement (skirmish-mode 2x/slow-mo for single-player). Not in v1.
- **Stateful AI hand-off on host migration.** Flagged; v1 restarts AI from current sim state on new host.
- **Custom paint tool in SeinARTSEditor.** Deferred to editor-module pass.

Changelog

- **2026-04-17** — Initial scaffold, cross-cutting invariants locked, 15 sections staked out. Abilities partially drafted (arbitration model and latent cleanup already implemented during the preceding cleanup chapter).
- **2026-04-17** — Q&A round 1: Entities and Tags locked. Key outcomes: single entity pool with optional transform, implicit `FSeinTagData` on every entity (initial tags sourced from

`USeinArchetypeDefinition::BaseTags`, authored in one place), refcounted tag presence, `Neutral` player as the owner sentinel, deletion of the `USeinTagsComponent` wrapper AC, and simplification of `USeinAbility::OwnedTags` to plain grant/ungrant (refcount replaces the diff-on-activate workaround).

- **2026-04-17** — Q&A round 2: Commands locked, CommandBrokers added as a new §4 primitive (16 sections total). Key outcomes: txn log carries pre-resolution intent (not post-resolution ability activations); `CommandType` is a gameplay tag rather than a C++ enum (designer-extensible, version-stable replays); common fields + optional `FInstancedStruct Payload` for per-type data; observer commands logged-but-non-sim with a hard "no sim influence" rule; one flat log; sim-side validation with `CommandRejected` visual events for UX. CommandBroker primitive subsumes formations / multi-unit dispatch / shift-queue / smart resolution: self-spawning and self-culling sim entities that mediate one player command to a dynamic member set, with pluggable resolvers via plugin settings, centroid + anchor transform semantics, and strict one-broker-per-member membership. Control groups remain a pure client-side selection preset, orthogonal to brokers.
- **2026-04-17** — Q&A round 3: Resources locked. Key outcomes: stockpile-only (flow is explicit non-goal), tag-keyed resources in `SeinARTS.Resource.*` namespace, hybrid resource authoring (plugin-settings catalog + per-faction kits), designer-configurable cap/overflow/spend behavior per resource with clamp-and-reject defaults, all four income sources supported (passive / entity / ability / trade), all three upkeep patterns supported (flat drain / pop cap / income-rate modifiers), match-settings-controlled resource sharing (not plugin-wide), unified `FSeinResourceCost` for abilities and production, symmetric availability APIs. Surfaced **match settings** as a new cross-cutting design area requiring its own pass.
- **2026-04-17** — Q&A round 4: Abilities locked (fully, not drafting). Key outcomes: full activation flow with deterministic ordering (cooldown → arbitration → target-validation → CanActivate → affordability → cost-deduct → cancel-others → cooldown-start → OwnedTags → OnActivate); cost deducts on activation success with `bRefundCostOnCancel` default true; cooldown refund `bRefundCooldownOnCancel` default false; `CooldownStartTiming` enum (OnActivate / OnEnd) per-ability; hybrid declarative + BP target validation (MaxRange, ValidTargetTags, RequiresLineOfSight on the class, CanActivate BP as escape hatch); `ESeinOutOfRangeBehavior` enum (Reject / AutoMoveThen) with AutoMoveThen integrating with CommandBrokers; passives are actives that auto-activate at spawn with no cost/cooldown and no auto-revive when cancelled; DefaultCommands + FallbackAbilityTag + resolver move from archetype def to `FSeinAbilityData` as sole source of truth; ability upgrades use existing primitives (effects for stats, grant/revoke for replacement) with no new "upgrade" concept.
- **2026-04-17** — Q&A round 5: Effects locked. Key outcomes: single `FSeinActiveEffect` primitive covering instant / finite / infinite durations via a sentinel-encoded `Duration` field + optional `TickInterval` for periodic payloads; per-effect stacking rule (Stack/Refresh/Independent) with `MaxStacks` integer cap; `bRemoveOnSourceDeath` flag default false (effects persist past source death by default); conditional effects are emergent (condition sources drive apply/remove themselves — no new primitive); rich effect definitions add `GrantedTags` (refcounted per §2), `RemoveEffectsWithTag`, and BP hooks (OnApply/OnTick/OnExpire/OnRemoved); `USeinEffect : UObject` as Blueprintable authoring surface with struct instances in sim storage (mirrors the ability authoring pattern); three application paths supported (ability-applied / condition-source-applied / effect-from-effect); recursion handled by apply-batching at tick boundaries with a dev-mode count-threshold warning; archetype-scope effects use the full effect treatment via an unified `FSeinActiveEffect` list on `FSeinPlayerState`.
- **2026-04-17** — Q&A round 6: Production locked. Key outcomes: per-resource cost deduction timing (`AtEnqueue` vs `AtCompletion`) declared in the resource catalog; population / cap-bound resources via `CostDirection = AddTowardCap` with spawn-stall-at-completion semantics for natural cap-bound queue behavior; build time modifiers pre-bake at enqueue (tech doesn't retroactively speed existing queues); progress-proportional refund as the new default, opt-in fixed-percentage override via `FSeinProductionRefundPolicy` (breaking change from flat-100%); FIFO queues with cancel-any-index ratified; `ProductionCancelled` and `ProductionStalled` visual events; rally target extended to entity-or-location with auto-move-on-spawn via CommandBroker; production is not building-only (any entity with `FSeinProductionData` can produce); continuous auto-requeue deferred to designer-scripted abilities.
- **2026-04-17** — Q&A round 7: Tech / Upgrades locked. **Key unification: tech is not a distinct primitive — it's an effect (§7) granted by research, scenario, ability, or map event.** Research completion calls `SeinApplyEffect(target, TechEffectClass)`; revocation calls `SeinRemoveEffect`. Same pattern handles

all granting paths. Other outcomes: per-tech `bRevocable` flag default false; `TechScope` enum `{Player, Team}` for grant propagation (orthogonal to modifier scope); tiers are emergent from prerequisite chains (no tier primitive); `ForbiddenPrerequisiteTags` added for mutually-exclusive patterns; `CanResearch` BlueprintNativeEvent escape hatch for complex eligibility; tech revocation affects forward-looking state only (existing units stay, archetype-scope stat mods revert); `TechRevoked` visual event; `USeinTechBPFL::SeinGetTechAvailability` for UI; tech stacking via the effect stacking system (no tech-specific plumbing). **`FSeinPlayerState::UnlockedTechTags` migrates to a refcounted tag container unified with §2's entity tag system; `FSeinPlayerState::ArchetypeModifiers` is deprecated — modifiers live on archetype effects, `ResolveAttribute` iterates the effect list.**

- **2026-04-17** — Effects + Tech scope revision (post-round-7 follow-up). **`ESeinModifierScope` extended to three values: `Instance`, `Archetype`, `Player`.** The new `Player` scope covers modifiers targeting player-state-level fields (resource income rates, caps, pop cap, upkeep rates) that aren't entity attributes. §7 Effects and §9 Tech updated to cover all three scopes; previous "tech = archetype-scope effect" framing was too narrow. `FSeinPlayerState` grows a `PlayerEffects` list alongside `ArchetypeEffects`. `ResolveAttribute` splits entity-attribute resolution from player-state-attribute resolution.
- **2026-04-17** — Q&A round 8: Combat locked. **Key reframe: combat is NOT a framework pipeline.** Rewrote the section from a prescriptive damage-resolution flow to a minimal convenience layer. Framework provides: `FSeinCombatData` with only Health + MaxHealth; `SeinApplyDamage`/`SeinApplyHeal`/`SeinApplySplashDamage` BPFLs that write pre-computed deltas (no math); `SeinMutateAttribute` generic escape hatch; `SeinRollAccuracy` optional PRNG convenience; death lifecycle hook that activates a `SeinARTS.DeathHandler`-tagged ability if present then destroys; tag-keyed `FSeinPlayerStatsData` with `SeinARTS.Stat.*` defaults + `SeinBumpStat` BPFL for designer-extensible post-match stats; `DamageApplied`/`HealApplied`/`Death`/`Kill` visual events; `FSeinWeaponProfile` starter template that framework never reads (aid only); `SeinARTS.DamageType.Default` as the only shipped damage-type tag. Explicit non-goals: damage types / armor model / accuracy system / resolution order / weapons as primitives / death state machine. Designers own combat identity; framework provides conveniences.
- **2026-04-17** — Q&A round 9: Vision / LOS / Fog locked. Key outcomes: per-player vision with runtime-mutable `Private`/`TeamShared` scope; VisionGroup abstraction to share bitfields across `TeamShared` teammates; per-cell per-VisionGroup u8 `EVNNNNNN` bitfield with 2 framework bits (Explored, Visible) + 6 designer-configurable layer bits (extensible to u16 for 14 layers); per-layer refcount arrays sized to declared layer count; stamp-delta update model with minimal per-source descriptor (PreviousCell + Radius + PerceptionLayers, no per-source cell-list storage); template cache for common (Radius, PerceptionLayers) combinations; emission + perception layer model for stealth/detection; `FSeinVisionData` + `FSeinVisionBlockerData` entity components; discrete grid raycasting with per-layer blocker sets; event-driven entity-visibility notifications (`EntityEnteredVision`/`EntityExitedVision`); render-side fog tick decoupled from sim (default 10 Hz); sim-authoritative command validity (no ghost filtering). Stamp-delta-refcount confirmed as industry-standard pattern (used by most production RTS); no radically different approach found that beats it under deterministic-lockstep constraints.
- **2026-04-17** — Post-round-9 scale/memory revision (big one). Recognized RTS cell-count scales span 3+ orders of magnitude (small-scale squad-tactical ~1M cells to massive-unit ~25M+ cells). Promoted two cross-cutting invariants: **(8) Scale-aware grid-backed data** (compact per-cell structures, per-map tag palettes, tile partitioning, designer-configurable cell sizes matching unit scale) and **(9) Batch-tick heavy systems** (`TickInterval` on `ISeinSystem`, default 4-tick batching for heavy work like vision stamping). Rewrote §11 Vision defaults: vision grid 4:1 coarser than nav by default; 4-bit packed refcounts (15-cap, sufficient for RTS); 4-tick batched updates; tile-partitioned source registry; memory now fits at all target scales. Wrote §12 Terrain from scratch: compact cell-struct variants via plugin settings (`FSeinCellData_Flat`/`_Height`/`_HeightSlope`, 4-8 bytes per cell); per-map u8 tag palette with 4 inline indices + sparse overflow; tile grid as cross-cutting infrastructure (32×32 cells per tile default); `FSeinMapTile` summary struct used by vision, spatial queries, pathfinding, terrain-tag queries; `USeinSpatialBPFL::SeinQueryEntitiesInRadius` as a framework-level utility; nav composition locked as binding requirement for nav refactor (HPA* for inter-region abstract pathing + regional flow fields for intra-region/group orders + CommandBroker resolver for formations + `USeinMovementProfile` subclasses for steering — four-layer composition works at both small-scale and massive-scale RTS ends of the spectrum); `FSeinFootprintData` for multi-cell entities; cover query returns tags + directional facing vector for designer

dot-product math; `FSeinCapturePointData` with pluggable `CaptureAbility` field; runtime terrain mutation BPFLs; placed-volumes + nav-derived authoring (custom paint tool deferred to editor pass); memory fits: small 1v1 ~3 MB, large competitive ~20 MB, massive-scale ~124 MB. Framework practical ceiling ~5000^2 cells regardless of physical map size; designers pick cell size matching unit scale.

- **2026-04-17** — Starter-content policy noted in intro. Framework stays genre-neutral; starter content matching specific RTS subgenres (squad-tactical, formation-heavy, economy-flow, etc.) ships as optional assets under clearly-labeled namespaces like `SeinARTS.Starter.SquadTactics.*`. Framework core never depends on starters; designers use, extend, replace, or ignore. Explicit IP-safety policy: no specific commercial-game references in docs or code.
- **2026-04-17** — IP-safety scrub of DESIGN.md + CLAUDE.md. All specific commercial-game names (and abbreviations) removed from design documents and replaced with genre-descriptive language ("squad-tactical", "massive-scale", "large-formation", "cap-bound spawn-stall pattern"). Source code comments and docs site content still contain some references; queued as a follow-up cleanup pass before shipping.
- **2026-04-17** — Q&A round 11: Relationships (containment/transport/garrison/attachment) locked. Hybrid primitive: base `FSeinContainmentData` (occupants, capacity, query filter, eject-on-death flag, visibility mode) + specialization components (`FSeinAttachmentSpec`, `FSeinTransportSpec`, `FSeinGarrisonSpec`). Orthogonal relationship axes: entity may have at most one attachment + one containment simultaneously (tree structure for chained containments). Destruction propagation configurable with on-eject and on-death effect hooks. Capacity model: integer capacity + entity-size + accepted-tag-query + `CanAccept` BP escape. Three visibility modes: `Hidden` (excluded from spatial queries), `PositionedRelative` (position from container + slot offset), `Partial` (hidden from targeting but participates in specific abilities). Attribution stats always count contained entities. Starter entry/exit abilities shipped; subgenre-specific starter content (garrison fire, crew weapon operation, retreat) under `SeinARTS.Starter.SquadTactics.*`. Optional deterministic visual-slot assignment for replay visual consistency. Command routing designer-authored; view-model pattern handles UI. Locked cleanly given the orthogonal-axes insight that broker membership + attachment + containment are three independent primitives.
- **2026-04-19** — Q&A round 12: Selection & control groups locked. Client-side-only, no sim-state. Observer command stream (`SelectionReplaced`/`Added`/`Removed`, `ControlGroupAssigned`/`AddedTo`/`Selected`) is authoritative for replay UI reconstruction. `bSelectable: bool` flag on entities (default true; projectiles and other non-interactable entities opt out). **Selection does NOT require ownership; command issuance DOES** — enemy entities can be selected for info-panel display but not orderable; controller filters to owned entities before emitting `ActivateAbility`. Transport/containment selection nuance: group move commands disembark contained members if their container isn't also in the group; skip them (transport carries) if the container is in the group. Helper `SeinResolveMoveableSelection` partitions the selection. Death-driven cleanup: client-side `USeinSelectionComponent` listens to `Death` visual events and removes dead handles from groups/selection; replay reconstructs by filtering original assignments against live entities at playback. Nested containment exposed via three BPFLs (immediate occupants, flat all-nested, hierarchical tree). Framework ships a starter `USeinSelectionComponent` with standard selection primitives; designer subclasses or replaces. AI doesn't select — commands carry handles directly. Spectator/POV replay modes are a replay-app concern, not framework. Strengthened broker ownership invariant: broker member set must be wholly owned by single player; broker primitive enforces at instantiation (not just caller-side filter); allied-command-sharing is match-settings territory.
- **2026-04-19** — Q&A round 13: AI locked, thin section. **Key reframe: strategic AI sits ABOVE the sim layer, like a human. It does NOT need to be deterministic** (runs on one designated host; emitted commands are what crosses the lockstep boundary). Unit-level micro-AI is just passive abilities (§6) — not a separate system. Framework provides thin `USeinAIController` base class with tick + sim-state query BPFLs (both all-entities for cheating AI and visible-per-player for fog-respecting AI) + command-emit BPFL. No prescribed architecture (designer picks BT / FSM / utility / GOAP / custom). No difficulty primitive. Host runs AI; on host migration or dropped-player takeover, new host spins up AI controllers from current sim state (stateful hand-off deferred). Debug logging to regular output log (never command txn log). Framework ships one starter `USeinAIController_Starter` BP under starter content as a playable example — not a blessed pattern. Replay just replays command stream; AI doesn't re-run.
- **2026-04-19** — Q&A round 14: Scenario events / cinematics locked as a thin section. **Key insight: scenarios are NOT a new primitive — they're abstract sim entities (§1) with ability components (§6).**

Scenario logic composes from existing primitives: passive tick-polling abilities + response abilities activated via `SeinActivateAbility` by external code. Thin utility layer adds: `bIsAbstract` flag on `USeinArchetypeDefinition` (ActorBridge hides + skips cosmetic routing); entity lookup via framework-level `USeinEntityLookupBPFL` (promoted below); `USeinScenarioBPFL` with `SeinSetSimPaused` + `SeinPlayPreRenderedCinematic` + `SeinEndCinematic`; `PlayPreRenderedCinematic`/`EndCinematic` visual events; `ESeinCinematicSkipMode` enum (Individual/VoteToSkip/HostOnly/NoSkip); vote-to-skip forward-references §17 voting. Multiple scenarios composable. Save/resume via command-stream replay; no explicit scenario state. No objective primitive — too varied across games. Editor factory split: rename `SeinActorFactory` → `SeinEntityFactory` (minimal); add `SeinUnitFactory` (starter unit kit). Retracted: `USeinScenario` UObject; `TriggerEventTags` on abilities. Starter example scenario under `SeinARTS.Starter.*`.

- **2026-04-19** — **Entity lookup promoted to framework-DNA level.** What started as a scenario convenience BPFL was elevated to a cross-cutting framework primitive after recognizing it's useful everywhere ("find all Riflemen", "find the HQ", "find entities with tag X in radius Y"). `USeinWorldSubsystem` now maintains two global indices: `EntityTagIndex: TMap<FGameplayTag, TArray<FSeinEntityHandle>>` (auto-indexed by the §2 tag-refcount system on every grant/ungrant) and `NamedEntityRegistry: TMap<FName, FSeinEntityHandle>` (singleton lookup). Added as cross-cutting invariant #10 in the preamble. `USeinEntityLookupBPFL` exposes: named register/lookup/unregister, tag-based set/first/count/has queries, tag-based queries filtered by spatial radius or player ownership, hierarchical tag-query matching for parent-tag matching. Memory overhead: 4 KB – 400 KB trivial at RTS scale; CPU is O(1) amortized for grant/lookup, O(entities_with_tag) for ungrant (acceptable; can swap to TSet if hot). §2 Tags updated with the global-index decision; §16 Scenario references the shared lookup infrastructure instead of shipping a scenario-only registry.
- **2026-04-19** — Q&A round 15 (final): Match Settings & match flow locked as §17. Captures everything that's "not a sim primitive but affects how a match plays out." Storage: `FSeinMatchSettings` USTRUCT (framework-shipped fields) + `USeinMatchSettings` UObject asset (lobby presets) + `FInstancedStruct CustomSettings` (designer extensions). Immutable after match start — snapshot into `USeinWorldSubsystem` at sim-start, read via BPFL. Match flow state machine: `Lobby → Starting → Playing → Paused → Ending → Ended`, command-driven transitions, visual events for scenario/UI subscription. Pre-match countdown + minimum match duration as match settings. Pause modes: `Tactical` (commands accumulate during pause, default) vs `Hard` (commands rejected); per-call override possible. Voting primitive locked as first-class sim state with `FSeinVoteState`, `StartVote`/`CastVote` commands, `OnVoteStarted`/`OnVoteProgress`/`OnVoteResolved` visual events, resolution modes (Majority/Unanimous/HostDecides/Plurality), auto-expiration. Victory is scenario-driven via `SeinEndMatch(winner, reason)` — framework provides zero prescribed victory enum. Spectator support via `bIsSpectator` flag + command filter. Host migration via `ESeinHostDropAction` enum (EndMatch/PauseUntilNewHost/AutoMigrate/AITakeover). Diplomacy primitive: `ESeinPlayerRelation { Enemy, Neutral, Allied }` with get/set BPFL + `bAllowMidMatchAllies` gate + `OnPlayerRelationChanged` event. Replay header: `FSeinReplayHeader` with framework version, map, seed, settings snapshot, player registrations, tick range. Framework's philosophy captured explicitly: enforces invariants at choke points (sim/render boundary, command validation, tag refcount, entity lifecycle, effect/ability lifecycle, component storage); composition above stays designer-freestyle. Game modes, victory conditions, objectives, lobby UI all live in designer code. `AGameModeBase`-compatible integration point. Flagged as future enhancements: sim-speed multipliers for skirmish, stateful AI hand-off on host migration, custom paint tool in editor.
- **2026-04-19** — Diplomacy redesigned: the initial `ESeinPlayerRelation { Enemy, Neutral, Allied }` enum was too restrictive for richer grand-strategy / 4X diplomacy models (cold war, non-aggression pacts, defensive alliances, trade agreements, vassalage, etc.). Replaced with a **directional per-pair `FGameplayTagContainer`** — same composability as §2 entity tags. Framework ships baseline vocabulary (`SeinARTS.Diplomacy.State.*` + `SeinARTS.Diplomacy.Permission.*`) used by core systems (§4 broker owner-scope, §10 friendly-fire filter, §11 vision sharing). Designers extend with arbitrary tags (`MyGame.Diplomacy.Treaty.NonAggressionPact`, etc.) without framework changes. Mutation via `FSeinCommand::ModifyDiplomacy` (command-driven, replay-deterministic). `OnDiplomacyChanged` visual event for subscribers. Parallel `DiplomacyTagIndex` for O(1) tag-based pair queries. `bAllowMidMatchDiplomacy` match-setting gate (rename from `bAllowMidMatchAllies`). Convenience BPFL

wrappers (`SeinDeclareWar`, `SeinDeclarePeace`, `SeinProposeTruce`) for baseline transitions; full richness via `SeinModifyDiplomacy`.

- **2026-04-19 — All 17 sections locked.** Design pass complete. Next phase: implementation passes batched against accumulated deltas + ongoing IP-safety scrub of source comments + docs site before ship.